# DEEP LEARNING

# Sumário

Como funciona o Deep Learning ..... 5
Introdução ..... 5
Considerações Iniciais ..... 6
Deep Learning: pré-requisitos e definições ..... 7
From Shallow to Deep ..... 9
Motivações para Redes Profundas ..... 14
A aprendizagem profunda dentro da aprendizagem de máquinas ..... 14
Tipos de imitação na aprendizagem profunda ..... 15
Aprendizado de máquina supervisionada ..... 16
Aprendizado de máquina não supervisionada ..... 16
Conceitos Básicos de Redes Neurais Artificiais ..... 16
Arquitetura Básica ..... 18
Treinamento Supervisionado ..... 21
Redes Autocodificadoras ..... 24
Arquitetura básica e variações ..... 25
Aplicações ..... 30
Redes Convolucionais ..... 31
Conceitos e Operações ..... 33
Arquitetura ..... 35
Aplicações ..... 37
Redes Recorrentes ..... 38
Arquitetura ..... 38
Aplicações ..... 40
Técnicas para Treinamento de Redes Profundas ..... 41
Pré-treinamento Não Supervisionado ..... 41
Uso de Unidades Lineares Retificadas ..... 42
Desligamento (Dropout) ..... 43
Normalização em Lote ..... 44
Estudo de Frameworks ..... 45
Frameworks, ferramentas e plataformas ..... 45

Frameworks Estudadas ..... 50
Theano ..... 50
TensorFlow ..... 51
Keras ..... 52
Microsoft CNTK ..... 52
Trabalho Desenvolvido ..... 53
Resultados e Discussão ..... 53
Estudo de Caso I ..... 58
CLASSIFICAÇÃO DE ESPÉCIES FLORESTAIS USANDO APRENDIZAGEM EM PROFUNDIDADE (DEEP LEARNING) ..... 58
INTRODUÇÃO ..... 58
METODOLOGIA ..... 59
FUNDAMENTAÇÃO TEÓRICA ..... 61
PROTOCOLO EXPERIMENTAL ..... 61
RESULTADOS ..... 62
CONCLUSÃO ..... 63
Referências ..... 64

# NOSSA HISTÓRIA

A nossa história inicia-se com a ideia visionária e da realização do sonho de um grupo de empresários na busca de atender à crescente demanda de cursos de Graduação e Pós-Graduação. E assim foi criado o Instituto, como uma entidade capaz de oferecer serviços educacionais em nível superior.

O Instituto tem como objetivo formar cidadão nas diferentes áreas de conhecimento, aptos para a inserção em diversos setores profissionais e para a participação no desenvolvimento da sociedade brasileira, e assim, colaborar na sua formação continuada. Também promover a divulgação de conhecimentos científicos, técnicos e culturais, que constituem patrimônio da humanidade, transmitindo e propagando os saberes através do ensino, utilizando-se de publicações e/ou outras normas de comunicação.

Tem como missão oferecer qualidade de ensino, conhecimento e cultura, de forma confiável e eficiente, para que o aluno tenha oportunidade de construir uma base profissional e ética, primando sempre pela inovação tecnológica, excelência no atendimento e valor do serviço oferecido. E dessa forma, conquistar o espaço de uma das instituições modelo no país na oferta de cursos de qualidade.

# Como funciona o Deep Learning

## Introdução

Nos últimos anos, técnicas de Aprendizado Profundo tem revolucionado diversas áreas de aprendizado de máquina, em especial a visão computacional. Isso ocorreu principalmente por dois motivos: a disponibilidade de bases de dados com milhões de imagens [8, 46], e de computadores capazes de reduzir o tempo necessário para realizar o processamento dessas bases de dados. Antes disso, alguns estudos exploraram a utilização de representações hierárquicas com redes neurais, tais como o Neocognitron de Fukushima [11] e a rede neural para reconhecimento de dígitos de LeCun [28]. Apesar desses métodos serem conhecidos pelas comunidades de Aprendizado de Máquinas e Inteligência Artificial, grande parte dos esforços dos pesquisadores de outras áreas tiveram como foco outras técnicas. Por exemplo, em Visão Computacional abordagens baseadas em características invariantes a escala, Bag-of-Features, pirâmides espaciais e abordagens baseadas em dicionários foram bastante utilizadas durante o final da década de 2000. Características baseadas em frequência foram comumente utilizadas pela comunidade de Processamento de Sinais, Áudio e Fala, enquanto métodos de Bag-of-Words e suas variantes foram explorados no Processamento de Linguagem Natural.

Talvez um dos principais marcos que atraiu a atenção das diferentes comunidades tenha sido a publicação do artigo e código-fonte que implementa a rede neural convolucional AlexNet [26]. Os resultados apontavam para uma indiscutível performance estatística de tais métodos para realizar classificação de imagem e, a partir desse ponto, o Aprendizado Profundo (do inglês Deep Learning (DL)) passou a ser aplicado a diversas áreas, em particular, mas não exclusivamente, as áreas de Visão Computacional, Processamento de Imagens, Computação Gráfica. Redes neurais convolucionais (do inglês Convolutional Neural Networks (CNN)), Deep Belief Networks, Restricted Boltzmman Machines e Autoencoders (AE) começaram a aparecer como base para métodos do estado da arte em diversas aplicações. A competição ImageNet [8] teve grande impacto nesse processo, começando uma corrida para encontrar o modelo que seria capaz de superar o atual campeão nesse desafio de classificação de imagens, além de segmentação de imagens, reconhecimento de objetos, entre outras tarefas.

De fato, técnicas de Aprendizado Profundo oferecem atualmente um importante conjunto de métodos para analisar sinais como áudio e fala, conteúdos visuais, incluindo imagens e vídeos, e ainda conteúdo textual. Entretanto, esses métodos incluem diversos modelos, componentes e algoritmos. A variedade de palavras-chave utilizadas nesse

contexto faz com que a literatura da área praticamente se torne uma nova linguagem: Mapas de Atributos, Ativação, Campos Receptivos, dropout, ReLU, MaxPool, softmax, SGD, Adam, FC, etc. Isso pode fazer com que seja difícil para que um leigo entenda e consiga acompanhar estudos recentes. Além disso, apesar do amplo uso de redes neurais profundas por pesquisadores em busca da resolução de problemas, há ainda uma lacuna no entendimento desses métodos: como eles funcionam, em que situações funcionam e quais suas limitações?

Nesse texto, iremos descrever as Redes Neurais Profundas (Deep Neural Networks), e em particular as Redes Convolucionais. Como o foco é na compreensão do funcionamento desse grupo de métodos, outros métodos de Aprendizado Profundo não serão tratados tais como Deep Belief Networks (DBN), Deep Boltzmann Machines (DBM) e métodos que utilizam Redes Neurais Recorrentes (do inglês Recurent Neural Networks (RNN)) e Reinforcement Learning. Para maiores detalhes sobre esses métodos, recomendase [10, 27, 47] como referências para estudos que utilizam DBNs e DBMs, e [57, 17, 40] para estudos relacionados a RNNs. Para o estudo de Autoencoders, recomendamos [2] e [14], e para o estudo de Redes Geradoras Adversariais (Generative Adversarial Networks, GANs) [15]. Acreditamos que a leitura desse material será de grande ajuda para o entendimento dos outros métodos citados e não cobertos por esse texto. A partir da seção seguinte iremos nos referir ao Aprendizado Profundo pelo seu termo em inglês Deep Learning (DL), visto que é esse o mais comumente utilizado mesmo na literatura em lingua portuguesa. Utilizaremos ainda as siglas em inglês pelo mesmo motivo, e.g. CNN.

## Considerações Iniciais

Em aprendizado de máquina, uma característica (feature) é uma propriedade mensurável de um fenômeno de interesse [1]. Nesse contexto, são úteis transformações de uma representação de dados para outra em um nível conceitual mais alto e que possa ser efetivamente aproveitada por algoritmos de aprendizado. Tradicionalmente, algoritmos de aprendizado de máquina pressupõem que o conjunto de dados a ser usado no treinamento foi previamente construído por meio de um processo de engenharia de características. Nesse processo, as características (i.e., atributos) componentes do conjunto de dados são manualmente selecionadas por um ser humano. A escolha de características ao mesmo tempo relevantes e discriminativas é fundamental para a geração de modelos de aprendizado adequados.

Entretanto, esse processo está sujeito a erros quando depende da existência de um especialista do domínio em questão, que possa selecionar adequadamente as

características para compor o conjunto de dados de treinamento. Por meio de técnicas de aprendizado de representação [2], é possível que o próprio algoritmo de aprendizado identifique automaticamente um subconjunto de características adequado. Aprendizagem profunda é o termo usado para denotar o problema de treinar redes neurais artificiais que realizam o aprendizado de características de forma hierárquica, de tal forma que características nos níveis mais altos da hierarquia sejam formadas pela combinação de características de mais baixo nível [3, 4, 5, 6, 7, 8]. Muitos problemas práticos têm sido resolvidos por redes neurais compostas de uma única camada intermediária [9]. De fato, o Teorema da Aproximação Universal [10] garante que redes neurais com uma única camada oculta são aproximadores universais, i.e., são estruturas capazes de aproximar qualquer função contínua, contanto que se possa definir a quantidade suficiente de neurônios nessa única camada intermediária.

## Deep Learning: pré-requisitos e definições

Os pré-requisitos necessários para entender como Deep Learning funciona, incluem conhecimentos básicos de Aprendizado de Máquinas (ML) e Processamento de Imagens (IP), em particular conhecimentos básicos sobre aprendizado supervisionado, classificação, redes neurais Multilayer Perceptron (MLP), aprendizado não-supervisionado, fundamentos de processamento de imagens, representação de imagens, filtragem e convolução. Como esses conceitos estão fora do escopo deste capítulo, recomenda-se [13, 1] como leituras introdutórias. Assume-se também que o leitor está familiarizado com Álgebra Linear e Cálculo, além de Probabilidade e Otimização — uma introdução a esses tópicos pode ser encontrada na Parte 1 do livro-texto de Goodfellow et al. sobre Deep Learning [14]. Ainda assim tentaremos descrever os métodos de forma clara o suficiente para que seja possível o entendimento mesmo com rudimentos dos pré-requisitos. Métodos que utilizam Deep Learning buscam descobrir um modelo (e.g. regras, parâmetros) utilizando um conjunto de dados (exemplos) e um método para guiar o aprendizado do modelo a partir desses exemplos. Ao final do processo de aprendizado tem-se uma função capaz de receber por entrada os dados brutos e fornecer como saída uma representação adequada para o problema em questão. Mas como DL se diferencia de ML? Vamos ver dois exemplos.

Exemplo 1 — classificação de imagens : deseja-se receber como entrada uma imagem no formato RGB e produzir como saída as probabilidades dessa imagem pertencer a um conjunto possíveis de classes (e.g. uma função que seja capaz de diferenciar imagens que contém cães, gatos, tartarugas e corujas). Assim queremos aprender uma função como $f(x) = y$, em que $x$ representa a imagem, e $y$ a classe mais provável para $x$.

Exemplo 2 — detecção de anomalias em sinais de fala : deseja-se receber como entrada um áudio (sinal) e produzir como saída uma representação desse sinal que permita dizer se a entrada representa um caso normal ou anômalo (e.g. uma função que permita responder para anomalias na fala de uma pessoa que indique alguma enfermidade ou sín drome). Assim, a função a ser aprendida estaria no formato f(x) = p, em que p representa a probabilidade de x ser um áudio anômalo.

Note que a definição e ambos os exemplos são iguais aos de aprendizado de máquina: o Exemplo 1 é um cenário de aprendizado supervisionado, enquanto o Exemplo 2 se trata de um cenário semi-supervisionado (podendo também ser não supervisionado a depender da base de dados).

De ML para DL : a diferença quando se trata de DL é como se aprende a função f(.). De forma geral, métodos não-DL, comumente referidos na literatura como “superficiais” ou “rasos” (o termo em inglês, que utilizaremos, é shallow) buscam diretamente por uma única função que possa, a partir de um conjunto de parâmetros, gerar o resultado desejado. Por outro lado, em DL temos métodos que aprendem f(.) por meio da composições de funções, i.e.:

$$f(\mathbf{x}) = f_L(\cdots f_2(f_1(\mathbf{x}_1))\cdots)),$$

onde cada função fl(.) (o índice l se refere comumente a uma “camada”, veremos mais a frente o significado desse termo) toma como entrada um vetor de dados xl (preparado para a camada l), gerando como saída o próximo vetor xl+1. Para sermos mais precisos, cada função faz uso de parâmetros para realizar a transformação dos dados de entrada. Iremos denotar o conjunto desses parâmetros (comumente uma matriz) por Wl , relacionado a cada função fl , e então podemos escrever:

$$f_L(\cdots f_2(f_1(\mathbf{x}_1, W_1); W_2)\cdots), W_L),$$

onde x1 representa os dados de entrada, cada função tem seu próprio conjunto de parâmetros e sua saída será passada para a próxima função. Na equação acima, temos a composição de L funções, ou L camadas.

Assim uma das ideias centrais em Deep Learning é aprender sucessivas representações dos dados, intermediárias, ou seja, os xl ,l = 1⋯L acima. Os algoritmos de

DL resolvem o problema de encontrar os parâmetros W diretamente a partir dos dados e definem cada representação como combinações de outras (anteriores) e mais simples [14]. Assim a profundidade (em termos das representações sucessivas) permite aprender uma sequência de funções que transformam vetores mapeando-os de um espaço a outro, até atingir o resultado desejado.

Por isso é de fundamental importância a hierarquia das representações: cada função opera sobre uma entrada gerando uma representação que é então passada para a próxima função. A hipótese em DL é a de que, se tivermos um número suficiente de camadas L, espaços com dimensionalidade alta o suficiente, i.e., o número de parâmetros W em cada função (iremos detalhar o que isso significa nas próximas seções), e dados suficientes para aprender os parâmetros Wl para todo l, então conseguiremos capturar o escopo das relações nos dados originais, encontrando assim a representação mais adequada para a tarefa desejada [6]. Matematicamente, podemos interpretar que essa sequência de transformações separa as múltiplas variedades (do ponto de vista geométrico, do termo inglês manifolds que nos dados originais estariam todas enoveladas

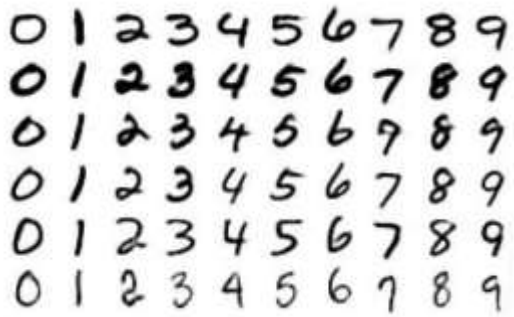


Figura 3.1. Exemplos de imagens das 10 classes da base de dígitos MNIST.

Na seção seguinte utilizaremos o exemplo do reconhecimento de dígitos para ilustrar os conceitos introduzidos, indo de um modelo shallow para um modelo deep

## From Shallow to Deep

Vamos tomar como exemplo o problema de classificar imagens contendo dígitos numéricos dentre 10 dígitos (de 0 a 9). Para isso utilizaremos a base de dados MNIST que possui imagens de tamanho 28 × 28 (veja exemplos na Figura 3.1). Montando uma arquitetura de rede neural simples, poderíamos tentar resolver o problema da seguinte forma:

1. entrada: vetorizamos a imagem de entrada 28×28 de forma a obter um vetor x de tamanho 784×1;

2. neurônios: criamos 10 neurônios de saída, cada um representando a probabilidade da entrada pertencer a uma das 10 classes (dígitos 0,1,2,3,4,5,6,7,8 ou 9);

3. pesos/parâmetros: assim como em uma rede MLP, cada neurônio de saída está associado a pesos e termos bias que são aplicados nos dados para gerar uma combinação linear como saída;

4. classificação: para que a saída possa ser interpretada como probabilidades permitindo atribuir um exemplo à classe com maior probabilidade, utilizamos uma função conhecida como softmax.

Vamos recordar a montagem de uma rede neural e definir a notação. Na nossa arquitetura shallow, cada neurônio j processa uma imagem de entrada na forma w t j x+bj . Para ilustrar, se tivermos uma entrada com 4 valores no vetor, o neurônio j produziria o seguinte resultado:

$$\mathbf{w}_j^t\mathbf{x} + \mathbf{b}_j = (w_{j,1}x_1 + b_1) + (w_{j,2}x_2 + b_2) + (w_{j,3}x_3 + b_3) + (w_{j,4}x_4 + b_4)$$

Como temos apenas uma camada precisamos que esse valor resultante do neurônio j seja o score da rede neural para a classe j. Assim, a partir do valor gerado pelo produto vetorial e somados os biases, aplicamos a função softmax, que é uma forma de obter valores normalizados para o intervalo 0-1 para cada classe c. A probabilidade de predizer y como pertencendo à classe c, dada uma imagem de entrada x, um conjunto de pesos, w, e ainda os termos bias, b, ambos relativos ao neurônio da classe c é definida como:

$$P(y = c|\mathbf{x}; \mathbf{w}_c; \mathbf{b}_c) = \text{softmax}_c(\mathbf{x}^t\mathbf{w}_c + \mathbf{b}_c) = \frac{e^{\mathbf{x}^t\mathbf{w}_c + \mathbf{b}_c}}{\sum_j |e^{\mathbf{x}^t\mathbf{w}_j + \mathbf{b}_j}|}.$$

Note que então temos primeiro uma combinação linear x twc + bc, depois exponenciamos esse valor, e dividimos pela soma da saída de todos os outros neurônios, de forma que a soma seja unitária, respeitando os critérios para uma distribuição de probabilidade. A função softmax é considerada uma função de ativação para classificação. Veremos posteriormente que existem diversas funções de ativação possíveis: cada qual adequada a um tipo de cenário. Na Figura 3.2 mostramos a arquitetura pretendida. A título de ilustração mostramos dois casos, no primeiro teríamos como entrada apenas um escalar, enquanto que, no segundo caso, tem-se um vetor como entrada.

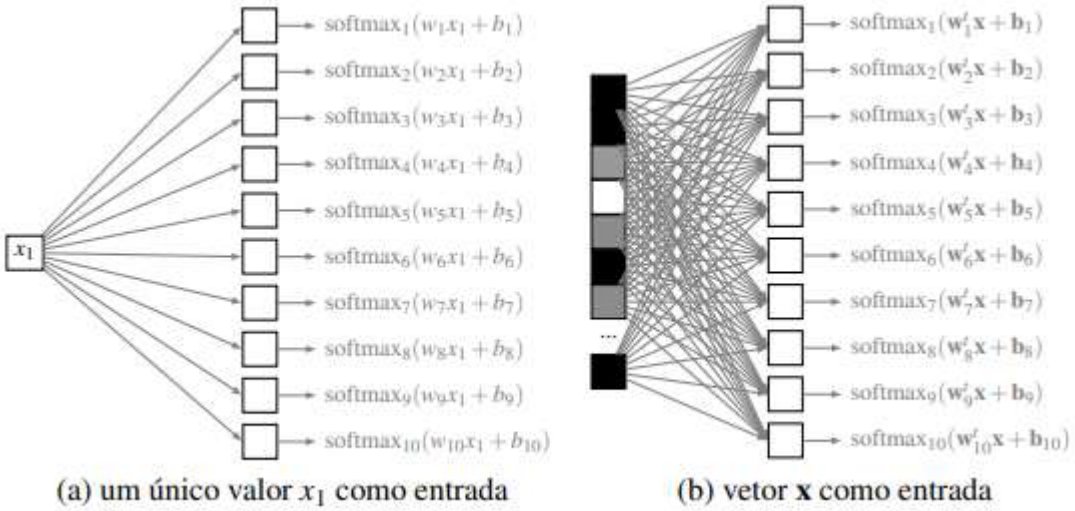



Figura 3.2. Uma arquitetura shallow: (a) caso em que teríamos apenas um valor escalar de entrada e 10 neurônios de saída, (b) mostra um vetor de valores como entrada, assim cada um dos 10 neurônios de saída está associado a um vetor de pesos diferente.

Na prática, ao invés de utilizarmos uma única imagem de entrada por vez, utilizamos um conjunto de exemplos de entrada, chamado de batch. Para que isso possa ser feito, utilizaremos notação matricial. Agora, o resultado da combinação linear computada pelos neurônios será dado por:

$$\mathbf{X} \cdot \mathbf{W} + \mathbf{b} \qquad (1)$$

X é uma matriz em que cada linha é uma imagem (ou seja, cada linha possui 784 valores, referentes aos 784 pixels da imagem). Aqui utilizaremos batches de tamanho 64, ou seja, a rede receberá como entrada 64 imagens de uma só vez e portanto a matriz X tem tamanho 64×784.

W é uma matriz que contém os pesos com os quais as imagens serão transforma das. Note que queremos produzir como saída, para cada imagem (linha da matriz X), as probabilidades para os 10 dígitos. Para isso a matriz W deverá ter 10 colunas e 784 linhas. Cada coluna contém os 784 pesos de um neurônio. b é um vetor de termos bias: são utilizados os mesmos biases para todas as 64 imagens no batch. Assim a Equação 1 pode ser detalhada da seguinte forma:

$$\begin{bmatrix} x_{0,0} & x_{0,1} & x_{0,2} & \cdots & x_{0,783} \\ x_{1,0} & x_{0,1} & x_{1,2} & \cdots & x_{0,783} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ x_{63,0} & x_{63,1} & x_{63,2} & \cdots & x_{63,783} \end{bmatrix} \cdot \begin{bmatrix} w_{0,0} & w_{0,1} & \cdots & w_{0,9} \\ w_{1,0} & w_{1,1} & \cdots & w_{1,9} \\ w_{2,0} & w_{2,1} & \cdots & w_{2,9} \\ \vdots & \vdots & \ddots & \vdots \\ w_{783,0} & w_{783,1} & \cdots & w_{783,9} \end{bmatrix} + \begin{bmatrix} b_0 \, b_1 \, b_2 \ldots b_9 \end{bmatrix}$$

Aplicamos então a função de ativação softmax para obter os resultados de classificação ou predições também em formato matricial Y. Note que essa será uma matriz com 64 linhas e 10 colunas, ou seja, para cada uma das 64 imagens no batch, temos as probabilidades para as 10 classes:

$$\mathbf{Y} = \text{softmax}(\mathbf{X} \cdot \mathbf{W} + \mathbf{b})$$

$$\mathbf{Y} = \begin{bmatrix} y_{0,0} & y_{0,1} & y_{0,2} & \cdots & y_{0,9} \\ y_{1,0} & y_{1,1} & y_{1,2} & \cdots & y_{1,9} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ y_{63,0} & y_{63,1} & y_{63,2} & \cdots & y_{63,9} \end{bmatrix}$$

Agora seria importante questionar: quando Y terá boas predições para as imagens de entrada, ou seja, predizendo corretamente os dígitos? Isso acontecerá apenas se os parâmetros da rede (pesos W e bias b) forem adequados para o sistema. Mas como encontrar esses parâmetros e como saber quão bons esses parâmetros são? Precisamos de uma função que compute a qualidade da predição realizada! Essa função é conhecida como função de custo (em inglês se utilizam os termos loss function ou cost function). Essa função é responsável por dizer quão longe estamos da predição ideal e portanto quantifica o “custo” ou “perda” ao aceitarmos a predição gerada pelos parâmetros atuais do modelo.

Em outras palavras, qual é o custo de aceitarmos uma predição ˆy sendo que a classe verdadeira é y? Para que possamos computar essa função precisamos nos basear em exemplos rotulados com suas classes verdadeiras, então o termo mais correto seria função de custo empírica. Dentre as funções de custo mais utilizadas em classificação temos a entropia cruzada (cross-entropy). Considerando um único exemplo cuja distribuição de probabilidade de classes real é y e a predição é dada por f(x) = yˆ temos a soma das entropias cruzadas (entre a predição e a classe real) de cada classe j:

$$\ell^{(\text{ce})} = -\sum_j y_j \cdot \log(\hat{y}_j + \varepsilon), \qquad (2)$$

onde ε << 1 é uma variável para evitar log(0). Vamos assumir ε = 0.0001.

A título de exemplo sejam os seguintes vetores de distribuição de probabilidade de classes (note que ambas classe real e predita são a classe 7):

$$\mathbf{y} = \begin{bmatrix} 0.00 & 0.00 & 0.00 & 0.00 & 0.00 & 0.00 & 1.00 & 0.00 & 0.00 & 0.00 \end{bmatrix}$$
$$\hat{\mathbf{y}} = \begin{bmatrix} 0.18 & 0.00 & 0.00 & 0.02 & 0.00 & 0.00 & 0.65 & 0.05 & 0.10 & 0.00 \end{bmatrix}$$

Então a entropia cruzada para esse exemplo seria:

$$\ell^{(\mathrm{ce})} = -(0+0+0+0+0+0-0.6214+0+0+0) = 0.6214$$

Essa função recebe como entrada um vetor de scores, e produz valor 0 apenas no caso ideal em que y = yˆ. Em problemas de classificação, a entropia cruzada pode ser vista como a minimização da divergência de Kullback-Leibler entre duas distribuições de classes na qual a distribuição verdadeira tem entropia nula (já que possui um único valor não nulo, 1) [42]. Assim, temos a minimização da log-verossimilhança negativa da classe correta, o que relaciona essa equação também com a estimação de máxima verossimilhança. Dado um conjunto atual de parâmetros W e b, o custo completo com base em um batch (ou conjunto de treinamento) contendo N instâncias é geralmente computado por meio da média dos custos de todas as instâncias xi e suas respectivas distribuições de classe: yi :

$$\mathscr{L}(\mathbf{W};\mathbf{b}) = \frac{1}{N}\sum_{i=1}^{N} \ell\left(\mathbf{y}_i, f(\mathbf{x}_i;\mathbf{W};\mathbf{b})\right).$$

Agora, precisamos ajustar os parâmetros de forma a minimizar L (W;b). Isso porque comumente esses parâmetros são inicializados com valores aleatórios, e queremos modificar esses valores iniciais de maneira a convergir para um modelo que nos dê boas predições. Esse processo é feito utilizando algoritmos de otimização como o Gradiente Descendente (GD), que basicamente computa derivadas parciais de forma a encontrar, para cada parâmetro do modelo, qual modificação dos parâmetros permite minimizar a função. Veremos nas seções seguintes os métodos de otimização comumente utilizados em Deep Learning. Por enquanto assumiremos que o leitor está familiarizado com o Gradiente Descendente (GD) básico. Assim, executamos o treinamento por meio do algoritmo conhecido por Backpropagation, que irá atualizar os parâmetros de forma que a saída yˆ se aproxime do resultado esperado y

## Motivações para Redes Profundas

Uma das motivações para a investigação de redes neurais artificiais compostas de muitas camadas veio da neurociência, mais especificamente do estudo da parte do cérebro denominada córtex visual [11]. Sabe-se que, quando algum estímulo visual chega à retina, o sinal correspondente percorre uma sequência de regiões do cérebro, e que cada uma dessas regiões é responsável por identificar características específicas na imagem correspondente ao estímulo visual. Em particular, neurônios das regiões iniciais são responsáveis por detectar formas geométricas simples na imagem, tais como cantos e bordas, enquanto que neurônios nas regiões finais têm a atribuição de detectar formas gráficas mais complexas, compostas das formas gráficas simples detectadas por regiões anteriores. Dessa forma, cada região de neurônios (que é análoga ao conceito de camada em redes neurais artificiais) combina padrões detectados pela região imediatamente anterior para formar características mais complexas. Esse processo se repete através das regiões até que os neurônios da região final têm a atribuição de detectar características de mais alto nível de abstração, tais como faces ou objetos específicos [2].

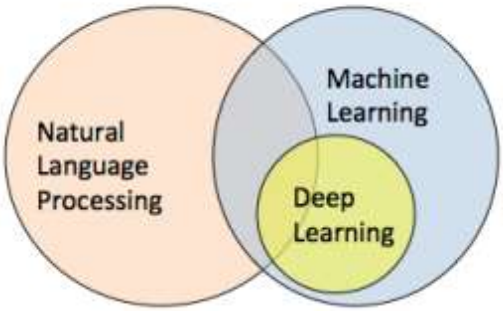
Há outra motivação, dessa vez teórica, para o estudo de redes profundas. Em [12], usando funções lógicas (boolean functions) como base para seu estudo, os autores demonstram que é possível representar qualquer função lógica por meio de um circuito lógico de uma única camada, embora essa representação possa requerer uma quantidade de unidades na camada intermediária que cresce exponencialmente com o tamanho da entrada. Eles também demonstraram que, ao permitir que o circuito cresça em quantidade de camadas intermediárias, essas mesmas funções lógicas problemáticas poderiam ser representadas por uma quantidade de unidades ocultas que cresce polinomialmente com o tamanho da entrada.

## A aprendizagem profunda dentro da aprendizagem de máquinas

Dentro da Inteligência Artificial temos a área de Aprendizagem de Máquina. [4] Esse é um campo da ciência da computação que evoluiu bastante a partir do estudo de reconhecimento de padrões (“pattern recognition”) e da teoria da aprendizagem computacional em inteligência artificial. A aprendizagem de máquina tem uma ligação “particular” com o Big Data pois é a base da análise preditiva do inglês (“predicitive analytics” or “predictive modelling”). Uma divisão mais especializada da Aprendizagem de máquina que norteia este trabalho é a Aprendizagem Profunda. [5].

Conforme pode ser observado na Figura 1, apresenta a relação entre Processamento de linguagem natural do inglês (Natural Language Processing - NLP) e a relação com

Aprendizagem de máquinas e Aprendizagem Profunda. Aprendizagem Profunda é uma das técnicas da área a Aprendizagem de máquinas. [16]




A ideia básica de que o software de Aprendizagem Profunda pode simular um grande arranjo de neurônios em uma "rede neural" artificial é bastante antiga e tem levado a tantas decepções como também a avanços. Mas por causa das melhorias nas fórmulas matemáticas e nos computadores cada vez mais importantes, os cientistas de computação agora podem aumentar o número de camadas de neurônios virtuais do que anteriormente. [6].

## Tipos de imitação na aprendizagem profunda

Os tipos de imitação na Aprendizagem Profunda são baseados no trabalho de [7]. As máquinas observam um padrão e tentam imitá-lo de alguma maneira. Podem ser definidas como imitação direta e indireta como um paralelo aos dois tipos principais de aprendizado de máquinas: aprendizado de máquinas supervisionado e aprendizado automático sem supervisão [20]. O aprendizado automatizado da máquina é a imitação direta de um padrão entre dois conjuntos de dados. Isto é, sempre tentando capturar um conjunto de dados de entrada e transformá-lo em um conjunto de dados de saída. Essa pode ser uma capacidade incrivelmente poderosa e útil. Os seguintes exemplos: (entrada de conjuntos de dados em negrito e conjuntos de dados de saída em itálico):

• Os pixels de uma imagem para detectar a presença ou ausência de um gato, pixel é a menor unidade de uma imagem digital.

• Os filmes que o usuário gostou para prever os filmes que o usuário gostaria de ver.

• As palavras escritas de alguém para prever se eles estão felizes ou tristes.

• Os dados de uma central climática prever a probabilidade de chuva.

• Os sensores do motor do carro para prever as configurações de sintonia ótimas. • Os dados de notícias para prever o preço das ações de amanhã.

• Um arquivo de áudio bruto para prever uma transcrição do áudio.

Todas estas são tarefas supervisionadas de aprendizagem de máquinas. Em todos os casos, a aprendizagem da máquina algoritmo está tentando imitar o padrão entre os dois conjuntos de dados de tal forma que pode usar um conjunto de dados para prever o outro. [7]

### Aprendizado de máquina supervisionada

A aprendizagem supervisionada é aquela que se destina a fornecer diretamente poder discriminativo para fins de classificação de padrões, frequentemente caracterizando as distribuições a posteriori de classes condicionadas aos dados visíveis. A aprendizagem supervisionada é um método para transformar um conjunto de dados em outro. [8] Por exemplo, se tivemos um conjunto de dados de "Preços de ações de segundafeira", que registrou o preço de cada ação em cada Segunda-feira nos últimos 10 anos, e um segundo conjunto de dados de "preços de ações de terça-feira" registrado ao longo de no mesmo período de tempo, um algoritmo de aprendizagem supervisionado pode tentar usar um para prever o de outros [7].

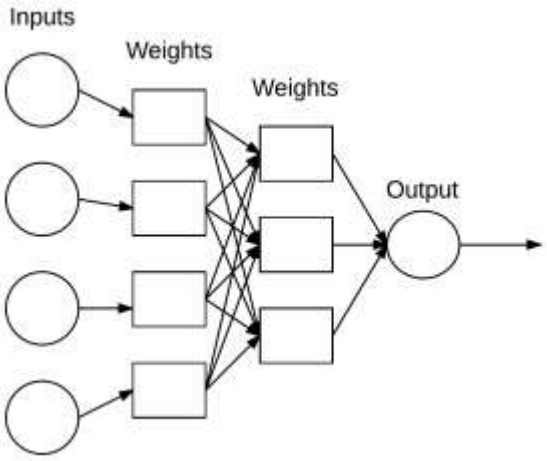
### Aprendizado de máquina não supervisionada

Já a aprendizagem não supervisionada se destina a capturar a correlação de alta ordem dos dados observados ou visíveis para fins de análise ou síntese de padrões quando não há informações sobre os rótulos das classes disponíveis na base. Aprendizagem de representação não supervisionada na literatura refere-se a essa categoria de redes profundas. Quando usado no modo generativo, também pode-se buscar a caracterização de distribuições estatísticas conjuntas dos dados visíveis e suas classes associadas quando disponíveis, podendo ainda estas serem tratadas como parte dos dados visíveis. [8]

## Conceitos Básicos de Redes Neurais Artificiais

As Redes Neurais Artificiais (RNA’s) [18] consistem em uma gama de ferramentas que possuem a capacidade de aprender e se adaptar para a realização de certas tarefas através de um processo de aprendizagem. As habilidades de aprendizagem e adaptação indicam que os modelos de redes neurais podem lidar com dados imprecisos e situações que não foram totalmente definidas no início do treinamento. Por isso, o uso da inteligência artificial se torna cada vez mais atrativa em técnicas de processamento de imagem, reconhecimento de padrão, problemas de classificação, controle de processos, entre outros. As RNA’s tiveram início na década de 40, pelo neurofisiologista MacCulloch, do Instituto de Tecnologia

de Massachussetts (MIT), e pelo matemático Waltter Pitts, da Universidade de Illinois, em que o trabalho realizado consistia em um modelo de resistores variáveis e amplificadores representando conexões sinápticas de um neurônio biológico. Desde então, diversos modelos de redes neurais têm sido desenvolvidos para aperfeiçoar e aumentar a aplicabilidade desta técnica em diversas áreas do conhecimento. [18]. Na Figura 2 é demonstrada uma arquitetura de rede neural simples.




Uma arquitetura de rede neural simples. As entradas são apresentadas à rede. Cada conexão carrega um sinal através das duas camadas ocultas na rede. Uma função final calcula o rótulo da classe de saída. [19] A classificação das técnicas de aprendizado profundo pode ser dividida em modelos discriminativos profundos ou em modelos generativos/nãosupervisionados [9]. Dentre os discriminativos, são elencadas as Redes Neurais Profundas do inglês (Deep Neural Network - DNN), Redes Neurais Recorrentes do inglês (Recurrent Neural Network - RNN), Redes Neurais Convolucionais do inglês (Convolutional Neural Network - CNN), dentre outros. Quanto aos modelos generativos, destacam-se a Máquina Restrita de Boltzmann do inglês (Restricted Boltzmann Machine - RBMs), Máquinas Profundas de Boltzmann do inglês (Deep Boltzmann Machines - DBM), Redes deep-belief, as redes auto- codificadoras regularizadas, etc. Entretanto, esse estilo de classificação bidirecional deixa escapar um conhecimento oriundo da pesquisa de aprendizagem profunda sobre como modelos generativos ou não supervisionados podem aprimorar bastante o desenvolvimento de DNN's e outros modelos de aprendizagem supervisionada através de melhor regularização ou otimização. Além disso, as redes profundas para a aprendizagem não supervisionada podem não necessariamente serem probabilísticas ou avaliar uma amostra significativa do modelo. [9]

Segundo Szegedy et al (2015), as redes neurais convolucionais estão no centro da maioria das soluções de visão computacional para uma ampla variedade de tarefas. Desde

2014, as redes convolucionais muito profundas começaram a se tornar convencionais, produzindo ganhos substanciais em vários pontos de referência. Embora o aumento do modelo e o custo computacional tendam a se traduzir em ganhos de qualidade imediatos para a maioria das tarefas (desde que dados suficientes sejam fornecidos para o treinamento) a eficiência computacional e a baixa contagem de parâmetros ainda são fatores facilitadores para vários casos de uso, como a visão móvel e cenários de grandes dados. Procurou-se modos de escalonar as redes de maneira que visem a utilização da computação adicionada, tão eficiente quanto possível, por convoluções fatorizadas [10].

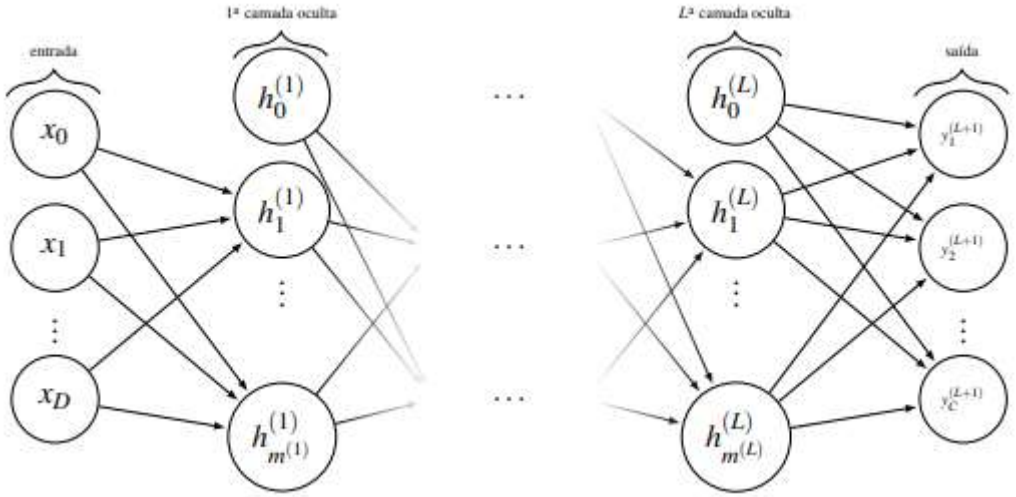
A arquitetura de uma rede neural diz respeito ao modo pelo qual suas unidades de processamento estão conectadas. Essa arquitetura influencia diretamente nos problemas que uma Rede Neural Artificial (RNA) pode resolver e na forma de realizar seu treinamento. Nesta Seção, apresentamos detalhes acerca da arquitetura básica de uma RNA, assim como descrevemos sucintamente aspectos do treinamento.

### Arquitetura Básica

A transmissão do sinal de um neurônio a outro no cérebro é um processo químico complexo, no qual substâncias específicas são liberadas pelo neurônio transmissor. O efeito é um aumento ou uma queda no potencial elétrico no corpo da célula receptora. Se este potencial alcançar o limite de ativação da célula, um pulso ou uma ação de potência e duração fixa é enviado para outros neurônios. Diz-se então que o neurônio está ativo. De forma semelhante à sua contrapartida biológica, uma RNA possui um sistema de neurônios artificiais (também conhecidas como unidades de processamento ou simplesmente unidades). Cada unidade realiza uma computação baseada nas demais unidades com as quais está conectada. Os neurônios de uma RNA são organizados em camadas, com conexões entre neurônios de camadas consecutivas. As conexões entre unidades são ponderadas por valores reais denominados pesos. A RNA mais simples possível contém uma única camada, composta por um único neurônio. Redes dessa natureza são bastante limitadas, porque resolvem apenas processos de decisão binários (i.e., nos quais há duas saídas possíveis) e linearmente separáveis (e.g., funções booleanas AND e OR). Por outro lado, é possível construir redes mais complexas (i.e., capazes de modelar processos de decisão não linearmente separáveis) por meio de um procedimento de composição de blocos de computação mais simples organizados em camadas.

A Figura 3.1 ilustra esquematicamente os elementos da arquitetura mais comum de RNA, uma Rede Alimentada Adiante (Feedforward Neural Network), ao mesmo tempo em

que apresenta parte da notação usada neste Capítulo. Nessa arquitetura, a rede forma um grafo direcionado, no qual os vértices representam os neurônios, e as arestas representam os pesos das conexões. A camada que recebe os dados é chamada camada de entrada, e a que produz o resultado final da computação é chamada camada de saída. Entre as camadas de entrada e saída, pode haver uma sequência de L camadas, onde ocorre o processamento interno da rede. Cada elemento dessa sequência é chamado de camada oculta. Após o seu treinamento (Seção 3.2.2), a rede é capaz de realizar um mapeamento de vetores no espaço D-dimensional para vetores no espaço C-dimensional.




Embora existam muitas convenções diferentes para declarar o número real de camadas em uma rede multicamada, neste Capítulo vamos considerar a quantidade de conjuntos distintos de pesos treináveis como o número de camadas de uma RNA. Por exemplo, considerando a Figura 3.1, quando L = 2, temos uma RNA de duas camadas de pesos: aqueles que ligam as entradas à camada oculta, e os que ligam a saída da camada oculta com as unidades da camada de saída.

Cada neurônio de uma camada oculta recebe um vetor x = (x1, x2,..., xN) como entrada e computa uma média ponderada pelos componentes de outro vetor de pesos associados w = (w1,w2,...,wN), em que cada componente pode ser interpretado como a força da conexão correspondente. É comum também considerar uma parcela adicional nessa média ponderada, correspondente ao denominado viés (bias), com peso constante e igual a +1 (correspondente a cada unidade h l 0 , 1 ≤ l ≤ L na Figura 3.1), cujo propósito é permitir que a rede neural melhor se adapte à função subjacente aos dados de entrada. Ao valor resultante dessa computação dá-se o nome de pré-ativação do neurônio (ver Eq. 1).

$$a(\mathbf{x}) = b + \sum_i w_i x_i = \mathbf{b} + \mathbf{w}^T \mathbf{x} \qquad (1)$$

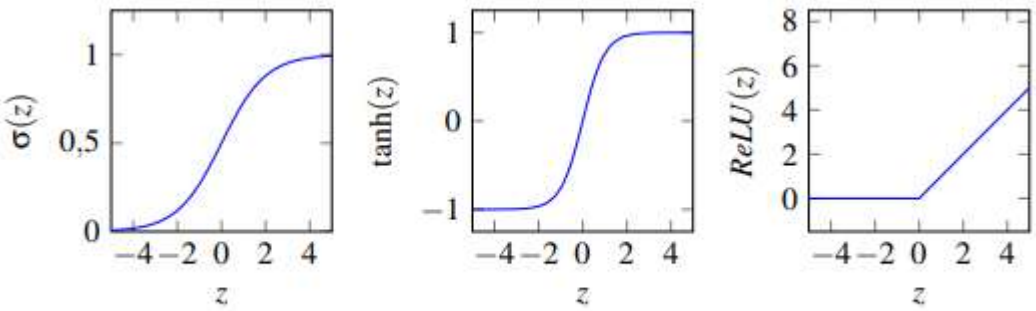
Após computar a(x), o neurônio aplica uma transformação não linear sobre esse valor por meio de uma função de ativação, g(·). A função composta h = g ◦ a produz o que se chama de ativação do neurônio (ver Eq. 2).

$$y = h(\mathbf{x}) = g(a(\mathbf{x})) = g(b + \sum_i w_i x_i) \qquad (2)$$

Há diferentes alternativas para a função de ativação g, dependendo da tarefa em questão. Três dessas funções são a sigmoide logística (Eq. 3), a sigmoide hiperbólica tangente (Eq. 4) e a retificada linear (Eq. 5).

$$g(z) = \frac{1}{1+e^{-z}} \quad (3) \qquad g(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} \quad (4) \qquad g(z) = \max(0, z) \quad (5)$$

A Figura 3.2 apresenta os gráficos dessas funções. Repare que, com exceção da ReLU em z = 0, todas são diferenciáveis e estritamente monotônicas. Repare também que as funções sigmoides possuem assíntotas que as limitam inferior e superiormente.




(a) Logística. (b) Hiperbólica tangente. (c) Retificada linear.

As equações previamente apresentadas para a pré-ativação e para a ativação de uma única unidade podem ser estendidas para o caso em que há L camadas ocultas, com vários neurônios em cada camada. Em primeiro lugar, em vez de pensar na pré-ativação de um único neurônio, podemos considerar a pré-ativação de toda uma camada oculta, conforme a Eq. 6. Nessa equação, que vale para todo k > 0, o sobrescrito k denota a k-ésima camada intermediária, e o uso do negrito indica que a Eq. 1 é aplicada a cada neurônio dessa camada. Explicação análoga pode ser apresentada para a Eq. 7. Além disso, W(k) é a matriz de pesos

das conexões entre os neurônios das camadas k − 1 e k. Finalmente, considere que, se k = 0, então h (k) (x) = x.

$$\mathbf{a}^{(k)}(\mathbf{x}) = \mathbf{b}^{(k)} + \mathbf{W}^{(k)}\mathbf{h}^{(k-1)}(\mathbf{x}) \tag{6}$$

$$\mathbf{h}^{(k)}(\mathbf{x}) = \mathbf{g}(\mathbf{a}^{(k)}(\mathbf{x})) \tag{7}$$

Como um todo, podemos considerar que, dado um vetor x de entrada, a RNA computa uma função f(x) tal que f(x) : R D → R C, conforme a Eq. 8.

$$f(\mathbf{x}) = \mathbf{h}^{(L+1)}(\mathbf{x}) = \mathbf{o}(\mathbf{a}^{(L+1)}(\mathbf{x})) \tag{8}$$

Na Eq. 8, o(·) representa a função de ativação usada para as unidades da camada de saída. Quando o propósito de uma RNA é realizar a tarefa de classificação, é comum selecionar para o(·) a função denominada softmax, que permite interpretar os valores da camada de saída como probabilidades posteriores. O valor produzido pela função softmax para a i-ésima unidade da camada de saída é dada pela Eq. 9.

$$o\left(a_i^{(L+1)}\right) = \frac{e^{a_i^{(L+1)}}}{\sum_{j=1}^{C} e^{a_j^{(L+1)}}} \tag{9}$$

As saídas das unidades a (L+1) i , 1 ≤ i ≤C, podem ser realmente interpretadas como valores de probabilidade, posto que pertencem ao intervalo [0,1] e sua soma é igual a 1.

## Treinamento Supervisionado

No aprendizado supervisionado, considera-se a disponibilidade de um conjunto de treinamento da forma {(x (t) ,y (t) ) : 1 ≤ t ≤ T}, em que x (t) é o vetor de características está associado ao componente y (t) . Considera-se que os elementos do conjunto de treinamento são independentes e identicamente distribuídos. No contexto de redes neurais, esse aprendizado (ou treinamento) supervisionado corresponde a utilizar o conjunto de treinamento para ajustar os valores dos parâmetros da rede, de tal forma que o erro de treinamento (training error) seja minimizado. Dessa forma, o treinamento de uma rede neural é convertido em um problema de otimização, cujo objetivo é minimizar o erro cometido pela rede, quando considerados todos os exemplos de treinamento

Com o propósito de medir o quanto a função inferida pela rede (Eq. 8) se ajusta aos exemplos de treinamento, o problema de otimização é definido para minimizar uma função de custo (cost function), J(f(x (t) ;θ),y (t) ), que mede a diferença (ou erro) que a rede comete relativamente ao exemplo x (t) . O uso de θ na definição dessa função denota a dependência de J com relação ao vetor de parâmetros θ da RNA. Duas alternativas para a escolha da função J são a soma dos erros quadrados (mean squared sum) e a entropia cruzada (cross entropy). A escolha da função de custo adequada depende da tarefa em particular. Detalhes acerca das propriedades de cada uma dessas funções podem ser encontrados em [1]. A Eq. 10 corresponde ao custo relativo ao conjunto de treinamento como um todo.

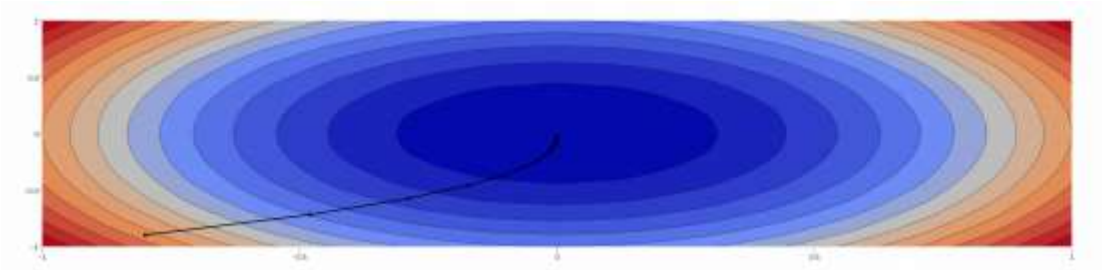
$$\mathbf{J}(\theta) = \underbrace{\frac{1}{T}\sum_{t=1}^{T} J(f(\mathbf{x}^{(t)};\theta),\mathbf{y}^{(t)})}_{\text{custo dos dados}} + \underbrace{\frac{\lambda}{2T}\sum_{k}\sum_{l} W_{k,l}^{2}}_{\text{custo de regularização}} \qquad (10)$$

Repare que a segunda parcela da Eq. 10 contém a soma dos quadrados de todos os pesos da RNA. Nessa parcela, o parâmetro λ, outro hiperparâmetro da RNA, controla a importância relativa entre as duas parcelas da função J(θ). Visto que o propósito da otimização é minimizar J(θ), podemos concluir que combinações de valores grandes para os pesos não são desejadas, ao menos que eles façam com que o valor da primeira parcela seja muito pequeno. A intuição aqui é que se o vetor de pesos ficar muito grande, um método de otimização que aplica pequenas alterações gradativas a esse vetor não terá condições de alterá-lo significativamente, resultando na convergência para uma solução inadequada. Além disso, incentivar o processo de otimização a manter os pesos pequenos resulta em um RNA menos suscetível a peculiaridades nos dados de entrada.

Quando θ é um vetor de muitas componentes (i.e., quando existem muitas funções candidatas) ou o conjunto de treinamento não é suficientemente grande, o algoritmo de treinamento é capaz de memorizar todos os exemplos de conjunto, sem produzir um modelo com boa capacidade de generalização. Esse fenômeno é conhecido como sobreajuste (overfitting). A segunda parcela da Eq. 10 é apenas uma forma, de diversas possíveis que podem ser usadas com o propósito de evitar que o processo de otimização fique preso em uma solução sub-ótima. Na Seção 3.6, apresentamos outras técnicas com esse mesmo propósito, que são genericamente conhecidas como técnicas de regularização.

Qualquer algoritmo de otimização pode ser utilizado para o treinamento de uma RNA. Entretanto, por razões práticas, um método de otimização comumente usado é o gradiente descendente (gradient descent, GD), ou uma de suas diversas variações (stochastic gradient descent, batch gradient descent, etc.). O GD considera a superfície multidimensional

representada pela função de custo J. A ideia básica desse procedimento é realizar (de forma iterativa) pequenas modificações no vetor de parâmetros θ que levem esse vetor na direção da maior descida nessa superfície, o que corresponde a seguir na direção do gradiente2 negativo da função, −∇J. Essa ideia pode ser ilustrada graficamente se considerarmos que J é uma função quadrática de apenas dois parâmetros, w1 e w2. Nesse caso, a Figura 3.3 apresenta uma curva de níveis de J (regiões mais escuras correspondem a valores cada vez menores de J). A sequência de segmentos de reta nessa figura representa o caminho formado pelas atualizações gradativas nos parâmetros w1 e w2 com o propósito de minimizar J.



Considere que w[t] corresponde ao vetor de pesos de uma RNA na t-ésima iteração do GD. É possível provar que a atualização nos parâmetros realizada pelo GD é dada conforme a Eq. 11. Nessa equação, γ é a taxa de aprendizagem (learning rate), um hiperparâmetro da RNA cujo valor adequado precisa ser encontrado. De acordo com o discutido em [1], esta abordagem tem várias dificuldades. Uma delas é a escolha do valor da taxa de aprendizagem que propicie o aprendizado rápido, mas ao mesmo tempo, evite o fenômeno da oscilação.

$$\Delta w[t] = -\gamma \frac{\partial J}{\partial w[t]} = -\gamma \nabla J(w[t]) \qquad (11)$$

Um aspecto importante na Eq. 11 diz respeito ao modo de calcular as derivadas parciais ∂l / ∂w[t] . Nesse contexto, um componente fundamental dos algoritmos para treinamento de redes neurais, incluindo as arquiteturas profundas modernas descritas adiante neste Capítulo, é a retropropagação de erros através das camadas ocultas, a fim de atualizar os parâmetros (pesos e viés) utilizados pelos neurônios [13]. Esse algoritmo, denominado retropropagação de erros (backward propagation of errors ou backpropagation)

é combinado com algum método de otimização (e.g., gradiente descendente), que atua para minimizar a função de custo da RNA.

A ideia básica subjacente ao algoritmo de retropropagação é como segue: se a saída produzida pela rede para uma determinada observação x (t) é diferente da esperada, então é necessário determinar o grau de responsabilidade de cada parâmetro da rede nesse erro para, em seguida, alterar esses parâmetros com o propósito de reduzir o erro produzido. Para isso, dois passos são realizados, conforme descrição a seguir.

Para determinar o erro associado à observação x (t) , inicialmente o algoritmo de retropropagação apresenta x (t) como entrada para a rede e propaga esse sinal através de todas as camadas ocultas até a camada de saída. Esse primeiro passo é denominado propagação adiante (forward step). Ao realizar esse passo, o algoritmo mantém o registro das pré-ativações a (k) e ativações h (k) para todas as camadas intermediárias, uma vez que estes valores serão utilizados para atualização dos parâmetros. O segundo passo (denominado passo de retropropagação) consiste em atualizar cada um dos pesos na rede de modo a fazer com que a saída produzida pela rede se aproxime da saída correta, minimizando assim o erro para cada neurônio de saída e para a rede como um todo. O procedimento composto dos dois passos acima descritos é resumido no Algoritmo 1. O algoritmo de retropropagação é uma aplicação da regra da cadeia para computar os gradientes de cada camada de uma RNA de forma iterativa.

## Redes Autocodificadoras

Redes autocodificadoras, também conhecidas como autocodificadoras ou redes autoassociativas, são uma classe de rede neural que pode ser treinada para aprender de forma não supervisionada características (features) a partir de um conjunto de dados. Estas características são úteis para uso posterior em tarefas de aprendizado supervisionado, tais como reconhecimento de objetos e outras tarefas relacionadas à visão computacional. Curiosamente, uma autocodificadora é normalmente treinada não para predizer alguma classe, mas para reproduzir na sua saída a própria entrada. Essa família de redes permite o aprendizado de representações mais concisas de um conjunto de dados. Nesta Seção, apresentamos a arquitetura básica das autocodificadoras, além de algumas variações. Descrevemos também algumas aplicações dessa família de redes.

**Algorithm 1** Algoritmo de aprendizado por retropropagação.

**for** $t \leftarrow 1, 2, \ldots, T$ **do**

PASSO ADIANTE

Com início na camada de entrada, usar a Eq. (6) e a Eq. (7) para propagar o exemplo $\mathbf{x}^{(t)}$ através da rede.

PASSO DE RETROPROPAGAÇÃO

Computar as derivadas da função $J$ com relação às ativações da camada $L+1$

**for** $l \leftarrow L, L-1, \ldots, 1$ **do**

Computar as derivadas da função $J$ com relação às entradas das unidades da camada $l+1$

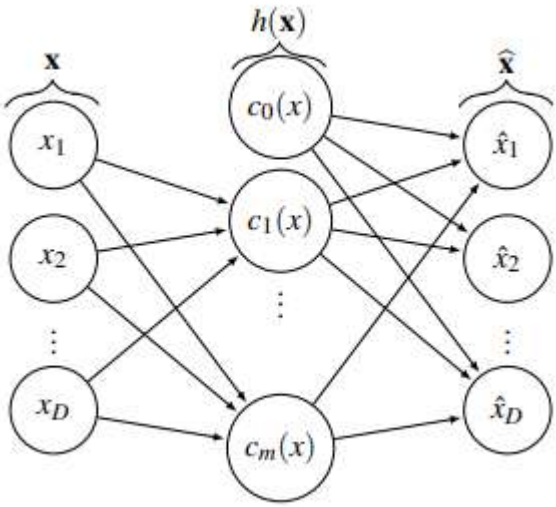
Computar as derivadas da função $J$ com relação aos pesos entre a camada $l$ e a camada $l-1$

Computar as derivadas da função $J$ com relação às ativações da camada $l-1$

**end for**

Atualizar os pesos.

**end for**




## Arquitetura básica e variações

Uma autocodificadora possui pelo menos uma camada intermediária, e as camadas de entrada e de saída possuem igual quantidade de unidades, D. Essa rede é treinada para tentar produzir na saída um padrão bx que seja o mais próximo possível do padrão de entrada x. Na Figura 3.4, que apresenta um esquema de uma rede autocodificadora com uma única camada intermediária, repare que x,bx ∈ IRD.

Uma rede autocodificadora realiza transformações na entrada de acordo com dois tipos de funções, conforme descrito a seguir.

• Função de extração de características, h : IRD → IRK . Essa função, conhecida como codificadora (encoder) mapeia as variáveis do conjunto de dados de treinamento para uma representação latente.

• Função de reconstrução, r : IRK → IRD. Conhecida como decodificadora (decoder), essa função mapeia a representação produzida por h de volta para o espaço original.

Uma escolha comum para as funções h e r (principalmente quando o vetor de entrada x é binário) é utilizar a função sigmoide (Seção 3.2.1) aplicada à transformação linear correspondente à multiplicação de x pela respectiva matriz de pesos: h(x) = σ(b1+ W1 · x) e bx = r(h(x)) = σ(b2 +W2 ·h(x)). Nessas equações, W1 e W2 são as matrizes de pesos das camadas intermediária e de saída, respectivamente. Outra decisão de projeto comum quando da definição da arquitetura de uma rede autocodificadora é usar o mecanismo de pesos atados (tied weights), o que significa que W2 = W0 1 (i.e., uma matriz é a transposta da outra). Essa decisão tem um efeito regularizador (Seção 3.2.2), porque resulta em menos parâmetros para otimizar (quando comparado ao caso em que se permite que W2 6= W0 1 , além de prevenir que soluções degeneradas resultem do processo de treinamento. De todo modo, a decisão de usar ou não pesos atados depende do conjunto de dados a ser usado no treinamento [].

Durante o treinamento de uma rede autocodificadora, o objetivo é definir um conjunto de parâmetros no par de funções h e r tal que o erro de reconstrução (i.e., a diferença entre x e bx) seja minimizado. Esse erro é representado pela função de custo L apresentada na Eq. 12. Dependendo da arquitetura da rede e da natureza do conjunto de treinamento, podem ser feitas diferentes escolhas para L(·). Para dados binários, pode ser usada a entropia cruzada (cross-entropy), enquanto que a média de erros quadrados (mean squared error) é popular para dados contínuos. Porque a saída de uma autocodificadora deve ser similar a sua entrada, ela pode ser treinada conforme discutido na Seção 3.2.2 (i.e., usando o algoritmo de retropropagação combinado com algum procedimento de otimização), com o cuidado de substituir os valores-alvo desejados (que não são conhecidos neste caso) pela própria entrada x.

$$L(\mathbf{X}) = \sum_{\mathbf{x}^{(i)} \in \mathbf{X}} \ell(\mathbf{x}^{(i)}, r(h(\mathbf{x}^{(i)}))) \qquad (12)$$

Idealmente, os parâmetros aprendidos durante o treinamento devem resultar em uma autocodificadora que identifique uma representação latente dos dados de entrada por meio de uma redução de dimensionalidade, ao mesmo tempo em que mantém o máximo de

informação acerca da distribuição de entrada. Considere que a representação latente descoberta por uma autocodificadora tenha dimensão D 0 . No caso em que D 0 < D, temos uma autocodificadora subcompleta (undercomplete autoencoder). Por outro lado, em uma autocodificadora supercompleta (overcomplete autoencoder), temos que D 0 > D. A escolha do valor de D 0 determina a quantidade de unidades da camada intermediária, assim como que tipo de informação a autocodificadora pode aprender acerca da distribuição de entrada.

No caso subcompleto, a autocodificadora deve aprender uma representação compacta da entrada. Como ilustração desse caso, considere um conjunto de dados de treinamento fictício contendo T exemplos. Considere ainda que, em cada exemplo, o esquema de codificação usado seja um bit por estado5 . Nesse caso, uma autocodificadora deveria aprender (de forma não supervisionada) a representar os T exemplos da entrada por meio de uma codificação binária compactada, com log2 T bits para cada exemplo.

Por outro lado, se considerarmos o aspecto de redução de dimensionalidade, o caso supercompleto parece não ter utilidade alguma. Entretanto, se o objetivo é encontrar características da entrada para posteriormente serem apresentadas a um classificador linear, quanto mais características houver, maior a probabilidade de que esse classificador tenha sucesso em identificar a classe correta de cada objeto. Entretanto, ao treinar uma autocodificadora supercompleta, é preciso tomar certas precauções para assegurar que a representação identificada seja útil. Um exemplo desse tipo de representação é a situação em que a autocodificadora apenas copia os D bits da entrada para D unidades na camada intermediária (deixando de usar D 0 −D unidades nessa camada). Variações da autocodificadora simples descrita acima foram propostas com o fim de evitar a produção de soluções indesejadas. Três dessas variações, descritas a seguir, são autocodificadora com filtragem de ruído (denoising autoencoder), autocodificadora contrativa (contractive autoencoder) e autocodificadora esparsa (sparse autoencoder).

Em uma autocodificadora com filtragem de ruído, o objetivo é fazer com que a representação aprendida seja robusta a ruídos nos dados de entrada. Isso pode ser realizado por meio da aplicação de um processo probabilístico em cada exemplo de treinamento antes de apresentá-lo à rede. Uma alternativa de processo probabilístico aplicável neste caso é atribuir zero em cada entrada de um exemplo, com probabilidade p, que passa a ser um hiperparâmetro da rede, denominado fator de ruído (noise factor). Outra possibilidade é

perturbar cada componente da entrada x por meio de um ruído gaussiano aditivo, com hiperparâmetros μ e σ (média e desvio padrão, respectivamente). Um aspecto importante é que, se x˜ for o resultado da aplicação do processo probabilístico ao exemplo x, a função de perda compara x e bx, ou seja, compara a saída com a entrada original, e não com x˜). A justificativa disso é fazer com que a rede treinada reconstrua a entrada original, ao mesmo tempo em que tenta inferir dependências entre as componentes que foram mantidas intactas e as que foram alteradas (zeradas) pela aplicação do processo probabilístico [14]. A Figura 3.5 apresenta exemplos passados para uma autocodificadora com filtragem de ruído treinada com 100 épocas, com ruído gaussiano, com uso de mini-lotes de tamanho igual a 128, e com fator de ruído p = 0,5.

Outra abordagem para coibir a produção de representações indesejadas é por meio do treinamento de uma autocodificadora contrativa [15]. Nessa abordagem, um termo é adicionado à função de perda para penalizar as representações indesejadas do espaço latente. Em particular, é adicionada uma parcela correspondente à norma de Frobenius da Jacobiana6 correspondente à função h com relação às unidades da camada de entrada. Veja a Eq. 13, composta de duas parcelas. A primeira incentiva o processo de otimização a encontrar soluções que sejam adequadas do ponto de vista do erro de reconstrução. Já a segunda parcela penaliza qualquer reconstrução, posto que se trata de uma função quadrada (cuja primeira derivada é zero). Sendo assim, a combinação dessas duas parcelas direciona o processo de otimização para manter apenas representações adequadas com relação ao erro de reconstrução

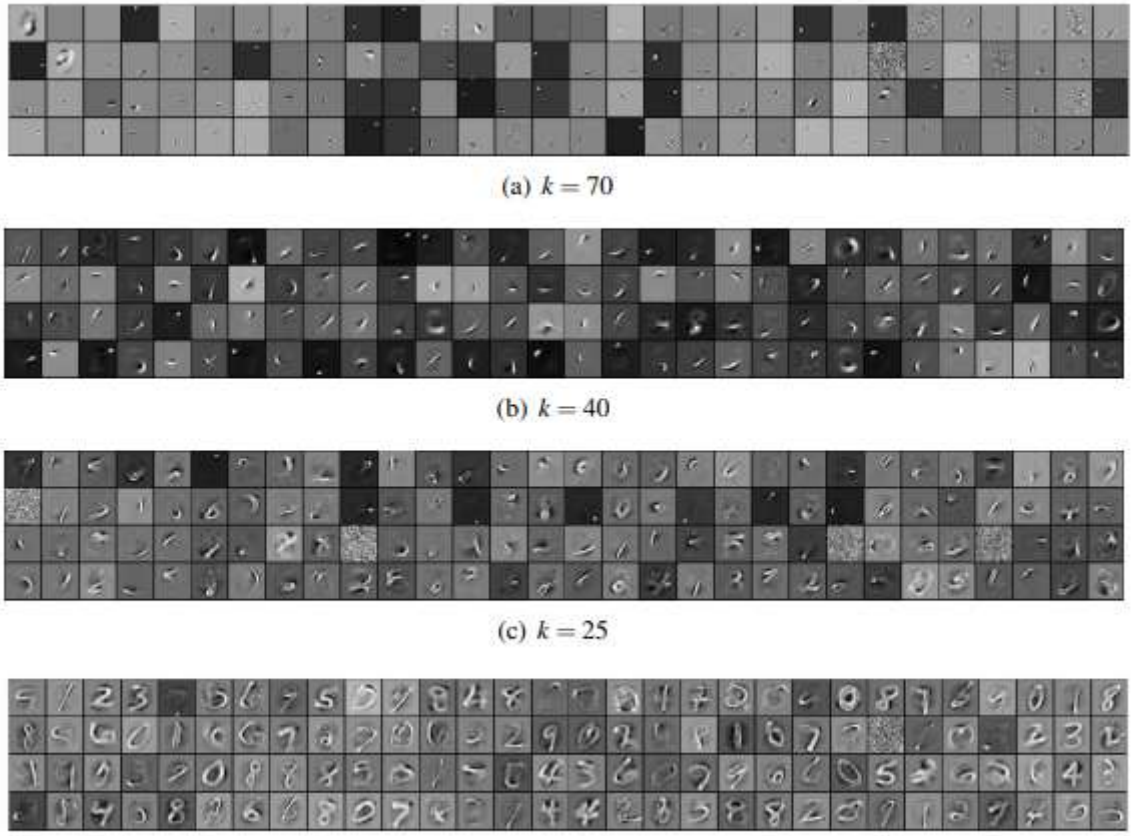
$$-\sum_{k=1}^{d}[x_k \log z_k + (1-x_k)\log(1-z_k)] + \|\frac{\partial h(x)}{\partial x}\|^2 \qquad (13)$$

Uma terceira alternativa para evitar que o algoritmo de treinamento aprenda uma representação inadequada (mesmo com o uso de mais unidades ocultas do que as existentes na camada de entrada) é por meio do uso das autocodificadoras esparsas, que procuram fazer com que apenas uma pequena quantidade de unidades da camada oculta seja ativada para cada padrão de entrada. Nesse tipo de rede, a esparsidade pode ser obtida por meio da adição de termos adicionais na função de perda durante o treinamento, ou mantendo apenas as k unidades mais ativas e tornando todas as demais unidades iguais a zero manualmente. Este último caso corresponde às denominadas autocodificadoras k-esparsas (k-sparse autoencoders), redes com funções de ativação lineares e pesos atados [16]. O

valor de k, um hiperparâmetro denominado fator de esparsidade, corresponde à quantidade de unidades ocultas que devem mantidas.

Uma vez que uma autocodificadora é treinada, pode ser útil visualizar (por meio de uma imagem bidimensional) a função codificadora h aprendida pela rede, com o propósito de entender que característica cada unidade oculta tenta detectar. Como exemplo desse processo, considere a situação de treinar uma autocodificadora k-esparsa de uma única camada intermediária de 1000 unidades sobre o conjunto de dados MNIST7 . A Figura 3.6 apresenta a visualização dos pesos após o treinamento para diferentes valores de k. Os pesos estão mapeados para valores de pixels de tal forma que um valor de peso negativo é preto, um valor de peso próximo de zero é cinza, e um valor de peso positivo é branco. Cada quadrado nessa figura apresenta o (norma limitada) da imagem de entrada x que ativa maximamente uma das 1000 unidades ocultas. É possível perceber que as unidades ocultas aprenderam a detectar bordas em diferentes posições e orientações na imagem. É possível também notar que, para k = 25 e k = 40, a saída é reconstruída usando um menor número de unidades ocultas do que em k = 70 e, portanto, as características tendem a ser melhores. Por outro lado, se o fator de esparsidade for muito pequeno (e.g., k = 10), as características detectadas são muito globais e inadequadas para fatorar a entrada em partes que façam sentido.

Outro tipo especial de autocodificadora, denominada autocodificadora linear (linear autoencoder), é obtida conforme descrito a seguir. Em primeiro lugar, deixamos de aplicar uma não linearidade para produzir a saída da camada intermediária, i.e., usamos $h(x) = b1 + W1 \cdot x$. Além disso, usamos a média de erros quadrados como função de perda L (ver Eq. 12). Pode ser provado [17, 18] que uma autocodificadora linear treinada com um conjunto de dados especialmente normalizado produz resultados equivalente aos obtidos por meio da técnica de Análise de Componentes Principais [19] e, por conta disso, é ótima no sentido de gerar o menor erro de reconstrução. Esse tipo de autocodificadora pode ser aplicado apenas no caso em que os dados de entrada residem em uma superfície multidimensional linear.



(a) $k = 70$

(b) $k = 40$

(c) $k = 25$

(d) $k = 10$

## Aplicações

Autocodificadoras são úteis no contexto do aprendizado de codificações mais compactas de um conjunto de dados, seja no sentido de menor dimensionalidade, ou no sentido de a representação resultante ser mais esparsa [20].

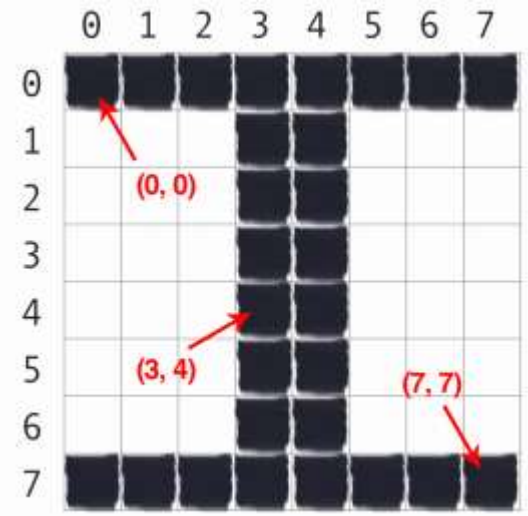
Autocodificadoras podem também ser usadas como uma espécie de bloco de construção para abordagens de aprendizagem profunda. Em particular, autocodificadoras podem ser usadas para iniciar os pesos de uma rede profunda para uso posterior em uma tarefa de aprendizado supervisionado. De fato, das duas abordagens originais de prétreinamento não supervisionado (Seção 3.6.1) para iniciar os pesos antes do treinamento supervisionado de uma rede profunda, uma delas envolvia o uso de autocodificadoras [21]. Em [22], os autores apresentam um estudo sobre o uso de autocodificadoras para pré-treinamento não supervisionado. De todo modo, autocodificadoras com esse propósito têm se tornado menos importantes nos últimos anos, muito pelos avanços obtidos em técnicas

alternativas para inicialização de pesos e para controlar a regularização de redes profundas (veja a Seção 3.6), assim como pela maior disponibilidade de quantidades suficientes de dados rotulados. Em [23], os autores apresentam o resultado do treinamento de uma autocodificadora esparsa de 9 camadas sobre um conjunto de 10 milhões de imagens. Eles conseguem demonstrar que é possível treinar um detector de faces sem a necessidade de usar um conjunto de treinamento no qual os exemplos estão rotulados.

### Redes Convolucionais

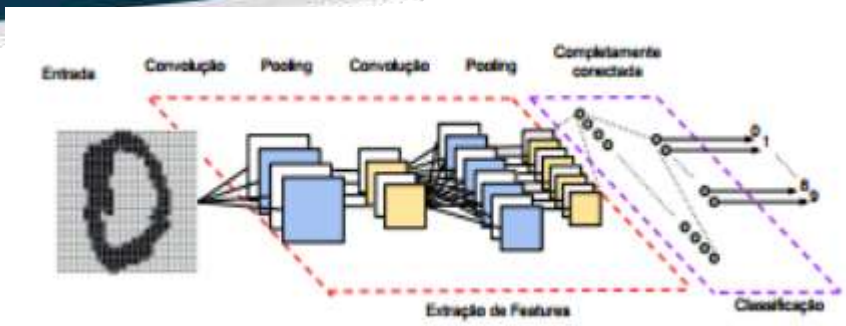
Redes neurais convolucionais (convolutional neural networks, CNN) se inspiram no funcionamento do córtex visual [24, 25]. Nesta Seção, apresentamos detalhes acerca dessa classe de redes. Descrevemos suas características particulares, tais como o compartilhamento de pesos e a conectividade esparsa. Apresentamos também as operações primitivas principais utilizadas em camadas intermediárias de redes dessa família, a saber, subamostragem e convolução.

A Rede Neural Convolucional é uma variante das Redes de Perceptrons de Múltiplas Camadas, tendo sido inspirada no método biológico de processamentos de dados visuais. De modo análogo aos processos clássicos de visão computacional, uma CNN (do inglês Convolutional Neural Networks) admite aplicar filtros em dados visuais, mantendo a relação de vizinhança entre os pixels da imagem ao longo do processamento da rede. A Figura 3 esboça uma matriz de pixel de uma imagem digital.




A letra "I" colocada em um pedaço de papel quadrado. Os pixels são acessados por suas coordenadas (x; y), onde x é a coluna para a direita e y para linhas [19].

A Figura 4 esboça uma CNN, este tipo de rede vem sendo vastamente utilizada, principalmente nas aplicações de categorização, detecção e importância em imagens e vídeos.




Uma rede neural convolucional incide em múltiplas partes com papéis diferentes. Inicialmente é comum aplicar sobre o dado de entrada camadas ditas de convolução. Uma camada de convolução é composta por múltiplos neurônios, cada um é responsável por aplicar um filtro em um pedaço característico da imagem. Podemos imaginar cada neurônio sendo conectado a um conjunto de pixels da camada anterior e que a cada uma dessa conexões se atribui um peso.

A combinação das entradas de um neurônio, utilizando os pesos concernentes de cada uma de suas conexões, gera uma saída ocorrida para a camada seguinte. Os pesos atribuídos às conexões de um neurônio podem ser decodificados como uma matriz que representa o filtro de uma convolução de imagens no domínio espacial (conhecido ainda como kernel ou máscara). Enquanto na formulação de Perceptrons clássica um neurônio é integrado a todos os neurônios da camada anterior, dito totalmente conectada, nas CNNs somente um subconjunto de entradas é conectado a cada neurônio. Com a transformação de arquitetura, as redes neurais convolucionais passam a realizar o diagnóstico de campos receptivos locais (local receptive fields). Os neurônios da mesma camada são coligados em mapas. Um mapa é produzido pelo agrupamento das saídas de neurônios que unidos cobrem uma parte da imagem que tenha sido processado com um filtro em comum. Para que um conjunto de neurônios de um determinado mapa utilize o mesmo filtro em distintas posições da imagem, é feito um compartilhamento dos pesos durante a técnica de treinamento.

Tal compartilhamento diminui expressivamente a quantidade de parâmetros a serem estudados e o tempo de treinamento da rede, por conseguinte. De modo a simplificar a esquematização dessas redes, um filtro a ser aplicado na imagem é constituído visualmente por somente um neurônio, causando a falsa sensação de que ele faria sozinho o procedimento de percorrimento da imagem. O tamanho do filtro determina a dimensão da vizinhança que cada neurônio da camada irá processar. Outra variável respeitável para a camada convolucional é o passo (ou stride). Este valor representa quantos pixels serão saltados entre cada janela, proferindo qual será a dimensão da camada subsequente na mesma unidade. Variavelmente do processo tradicional de Visão Computacional, no qual o

padrão inicial parte da definição de filtros (ou features) a serem empregadas, nas camadas de convolução das CNNs é necessário determinar somente a arquitetura dos filtros: quantidade e tamanhos, passos, dos filtros por camada. O método de aprendizado da rede transforma os pesos ao longo do treinamento, até localizar os melhores valores dos filtros para o conjunto de dados empregado. Uma das maneiras de se avaliar o aprendizado é constatar as ativações dos filtros adquiridos ao fim do treinamento.

### Conceitos e Operações

Redes de convolução se baseiam em algumas ideias básicas, a saber, campos receptivos locais (local receptive fields), compartilhamento de pesos (shared weights), convolução (convolution) e subamostragem (subsampling ou pooling). A seguir, apresentamos detalhes acerca desses conceitos. Redes neurais tradicionalmente são completamente conectadas, i.e., cada unidade na camada L está conectada a todas as unidades da camada L−1. Suponha que fôssemos modelar uma rede completamente conectadas com apenas uma camada intermediária para o conjunto de dados MNIST. Suponha ainda que essa única camada intermediária tenha o dobro de unidades da camada de entrada. Sendo assim, teríamos da ordem de 106 parâmetros para ajustar durante o treinamento dessa rede, e isso para imagens representadas em escala de cinza. (Para imagens coloridas, esse valor dever ser triplicado para considerar os 3 canais de cores, RGB.)

Além de possuir uma quantidade grande de parâmetros, a arquitetura de rede descrita no parágrafo anterior não leva em consideração a estrutura espacial tipicamente existente em dados correspondentes a imagens. Isso quer dizer que tanto pixels que estão próximos quanto os localizados em duas regiões muitos distantes da imagem (e.g., cantos superior esquerdo e inferior direito) são tratados indistintamente. Nessa arquitetura, a própria rede teria que detectar as dependências existentes na estrutura espacial da distribuição subjacente às imagens de entrada. Além disso, devido à conectividade muito grande, esse tipo de arquitetura sofre da denominada maldição da dimensionalidade (curse of dimensionality) e portanto não é adequado a imagens de alta resolução, por conta do alto potencial de sobreajuste (Seção 3.2.2) aos dados. Isso sem considerar o tempo para computar as pré-ativações de todas as unidades em cada camada.

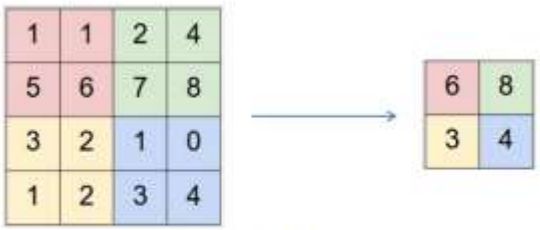
Em vez da conectividade global, uma rede convolucional utiliza conectividade local. Por exemplo, considere que L (1) é a primeira camada intermediária de uma CNN. Cada unidade de L (1) está conectada a uma quantidade restrita de unidades localizada em uma região contígua da camada de entrada L (0) , em vez de estar conectada a todas as unidades. As unidades de L (0) conectadas a uma unidade de L (1) formam o campo receptivo local dessa unidade. Por meio de seu campo receptivo local, cada unidade de L (1) pode detectar características visuais elementares, tais como arestas orientadas, extremidades, cantos. Essas características podem então ser combinadas pelas camadas subsequentes para detecção de características mais complexas (e.g., olhos, bicos, rodas, etc.).

Por outro lado, é provável que um determinado detector de alguma característica elementar seja útil em diferentes regiões da imagem de entrada. Para levar isso em consideração, em uma CNN as unidades de uma determinada camada são organizadas em conjuntos disjuntos, cada um dos quais é denominado um mapa de característica (feature map), também conhecido como filtro. As unidades contidas em um mapa de características são únicas na medida em que cada um delas está ligada a um conjunto de unidades (i.e., ao seu campo receptivo) diferente na camada anterior. Além disso, todas as unidades de um mapa compartilham os mesmos parâmetros (i.e., a mesma matriz de pesos e viés). O resultado disso é que essas unidades dentro de um mapa servem como detectores de uma mesma característica, mas cada uma delas está conectada a uma região diferente da imagem. Portanto, em uma CNN, uma camada oculta é segmentada em diversos mapas de características (os planos ilustrados na Figura 3.8), em que cada unidade de um mapa tem o objetivo realizar a mesma operação sobre a imagem de entrada, com cada unidade aplicando essa operação em uma região específica dessa imagem.

Cada unidade em um mapa de característica realiza uma operação denominada convolução. Para entender essa operação, considere que a imagem de entrada contém 28 × 28 pixels. Por simplicidade, vamos considerar ainda que essas imagens de entrada são em escala de cinza. Em processamento de imagens, uma convolução sobre uma imagem I corresponde a aplicar o produto de Hadamard8 entre a matriz de pixels de I e outra matriz, denominada núcleo da convolução (convolution kernel). Por exemplo, a Figura 3.7(a) apresenta o resultado da aplicação de um tipo particular de convolução sobre uma imagem. Outra operação importante utilizada em uma CNN é a subamostragem. Em processamento de imagens, a subamostragem de uma imagem envolve reduzir a sua resolução, sem no entanto alterar significativamente o seu aspecto. No contexto de uma CNN, a subamostragem reduz a dimensionalidade de um mapa de característica fornecido como entrada e produz outro mapa de característica, uma espécie de resumo do primeiro. Há

várias formas de subamostragem aplicáveis a um mapa de característica: selecionar o valor máximo (max pooling), a média (average pooling) ou a norma do conjunto (L2- pooling), entre outras. Como exemplo, considere que a ilustração à esquerda da Figura 3.7(b)9 corresponde às ativações das unidades de um mapa de característica de tamanho 4x4. A ilustração à direita apresenta outro mapa de característica, resultante da operação de subamostragem com o uso de filtros de tamanho igual a 2x2 e tamanho do passo igual a 2. O uso da subamostragem faz com que uma CNN seja robusta em relação à localizações exatas das características na imagem, uma propriedade que permite a esse tipo de rede aprender representações invariantes com relação a pequenas translações.

(a) Resultado de uma convolução (direita) aplicada a uma imagem (esquerda).

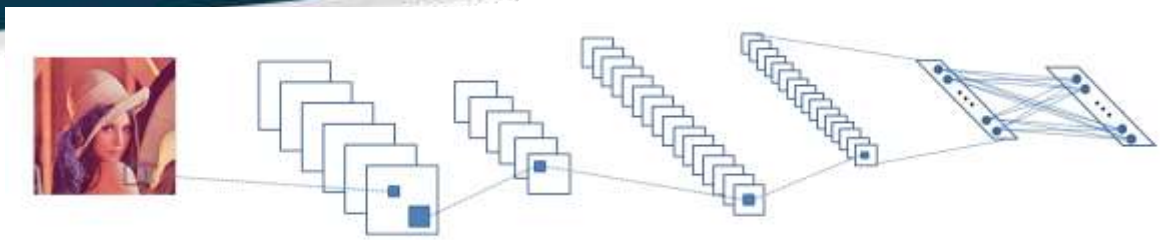



(b) Subamostragem com filtro de tamanho 2x2 e tamanho do passo igual a 2.

## Arquitetura

Os conceitos apresentados na Seção 3.4.1 servem de blocos de construção para montar as camadas de uma CNN, conforme descrevemos nesta Seção. Em geral, uma CNN possui diversos tipos de camadas: camadas de convolução, camadas de subamostragem, camadas de normalização de contraste e camadas completamente conectadas. Na forma mais comum de arquitetar uma CNN, a rede é organizada em estágios. Cada estágio é composto por uma ou mais camadas de convolução em sequência, seguidas por uma camada de subamostragem, que por sua vez é seguida (opcionalmente) por uma camada de normalização. Uma CNN pode conter vários estágios empilhados após a camada de entrada (que corresponde à imagem). Após o estágio final da rede, são adicionadas uma ou mais camadas completamente conectadas.

A Figura 3.8 apresenta um exemplo esquemático de uma CNN na qual, após a camada de entrada (que corresponde aos pixels da imagem), temos uma camada de convolução composta de 6 mapas de características (representados como planos na figura), seguida de uma camada de subamostragem, completando o primeiro estágio. Essa figura ainda ilustra um segundo estágio antes das duas camadas completamente conectadas.



No contexto de uma CNN, a computação realizada em cada mapa de característica M de uma camada de convolução C envolve a aplicação de várias operações de convolução, uma para cada unidade contida em M, conforme descrito a seguir. Cada unidade u de M realiza uma convolução usando a matriz de pesos de M. (Lembre-se de que todas as unidades em M compartilham a mesma matriz de pesos.) A matriz de pesos corresponde ao núcleo dessa convolução, e a matriz sobre a qual a convolução é aplicada correspondente ao campo receptivo local de u. A quantidade de unidades é a mesma em cada mapa de característica de C e depende (1) do tamanho do campo receptivo das unidades e (2) do denominado tamanho do passo (stride size). Esse segundo hiperparâmetro é um inteiro positivo (normalmente igual a 1) que define o quanto de sobreposição há entre os campos receptivos locais de duas unidades vizinha em um mapa L. Por exemplo, considere que as imagens usadas como entrada para C são de tamanho 28x28 pixels, que o campo receptivo local de cada unidade tem tamanho 5x5, e que o tamanho do passo seja igual a 1. Então, cada mapa de características em C deve conter 24x24 unidades. (A melhor maneira de entender esse resultado é imaginar que uma janela (de mesmo tamanho dos campos receptivos) deslize da esquerda para a direita e de cima para baixo na imagem de entrada.)

Embora a descrição aqui fornecida sobre o funcionamento de uma camada de convolução tenha sido apresentada considerando que C recebe uma imagem como entrada, o mesmo procedimento de funcionamento se aplica na computação realizada pelas demais camadas de convolução da CNN. Outro tipo de camada em uma CNN é a denominada camada de subamostragem (subsampling layer ou pooling layer). O objetivo dessa, que também é composta por um conjunto de mapas de características, é realizar a agregação das saídas (ativações) de um conjunto de unidades da camada anterior. É possível mostrar que camadas de subamostragem resultam em uma rede mais robusta a transformações espaciais (e.g., mais invariante a eventuais translações dos objetos em uma imagem). Redes neurais convolucionais também podem conter as denominadas camadas de normalização de contraste local (Local Contrast Normalization, LCN) [26]. Quando utilizada, essa camada é posicionada na saída da camada de subamostragem. Uma camada LCN normaliza o contraste de uma imagem de forma não linear. Em vez de realizar uma normalização global (i.e., considerando a imagem como um todo), a camada LCN aplica a normalização sobre

regiões locais da imagem, considerando cada pixel por vez. A normalização pode corresponder a subtrair a média da vizinhança de um pixel particular e dividir pela variância dos valores de pixel dessa vizinhança. Esta transformação equipa a CNN com a invariância de brilho, propriedade útil no contexto de reconhecimento de imagens.

O outro tipo de camada em uma CNN é a completamente conectada. Em arquiteturas de CNNs modernas, é comum encontrar uma ou duas camadas desse tipo antes da camada de saída. Juntamente com as camadas de convolução e subamostragem, as camadas totalmente conectadas geram descritores de características da imagem que podem ser mais facilmente classificados pela camada de saída (que normalmente usa a função softmax; ver Seção 3.2.1). É importante notar que os pesos dos mapas de características em cada camada de convolução são aprendidos durante o treinamento. Por conta de as operações de convolução e subamostragem serem diferenciáveis, uma CNN pode ser também treinada por meio do algoritmo de retropropagação (Seção 3.2.2), com a diferença de que os pesos são atualizados considerando a média dos gradientes dos pesos compartilhados. Durante esse treinamento, é comum aplicar a técnica de desligamento (Seção 3.6.3) às camadas completamente conectadas para evitar o sobreajuste.

### Aplicações

Por meio das redes neurais convolucionais, é possível realizar a detecção de padrões em imagens de forma mais adequada, no sentido de que características dos dados dessa natureza podem ser exploradas para obter melhores resultados. Desde sua origem, esse tipo de rede tem se mostrado adequado em tarefas que envolvem reconhecimento visual, tais como reconhecimento de caracteres manuscritos e detecção de faces [27, 7, 24]. Com o sucesso resultante do uso de arquiteturas profundas para o processamento visual e o surgimento de bases de dados de imagem com milhões de exemplos rotulados (por exemplo, ImageNet, Places), o estado da arte em visão computacional por meio de CNNs tem avançado rapidamente.

Essas redes ganharam popularidade em 2012, quando foram usadas para reduzir substancialmente a taxa de erro em uma conhecida competição de reconhecimento de objetos, a ImageNet12 [28, 29]. Conforme apresentado nesta Seção, o objetivo de cada mapa de característica em uma CNN é extrair características da imagem fornecida. Portanto, uma CNN também desempenha o papel de extrator automático de características da imagem. De fato, desde 2012, a área de Visão Computacional tem sido invadida por abordagens que usam CNNs para tarefas de classificação e detecção de objetos em imagens, em substituição às abordagens anteriores que envolviam a extração manual das

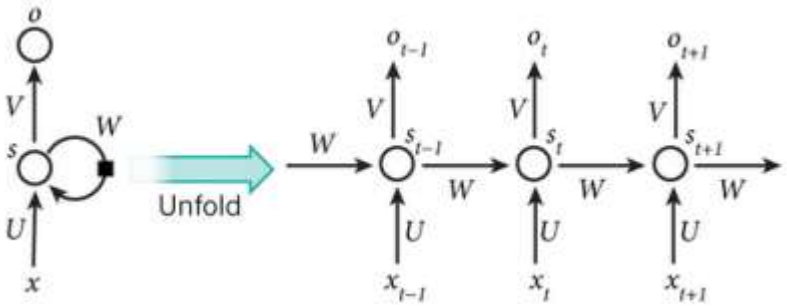
características das imagens para posteriormente aplicar algum método de classificação, como por exemplo as máquinas de vetores suporte (support vector machines, SVM). Aplicações mais recentes de CNNs envolvem a detecção de pedestres [30] e de placas de sinais de trânsito [31].

## Redes Recorrentes

Redes Neurais Recorrentes (Recurrent Neural Networks, RNN) constituem uma ampla classe de redes cuja evolução do estado depende tanto da entrada corrente quanto do estado atual. Essa propriedade (ter estado dinâmico) proporciona a possibilidade de realizar computação dependente do contexto e aprender dependências de longo prazo: um sinal que é fornecido a uma rede recorrente em um instante de tempo t pode alterar o comportamento dessa rede em um momento t +k, k > 0.

### Arquitetura

A arquitetura usada em RNNs é adequada para permitir o processamento de informação sequencial (e.g., textos, áudio e vídeo). Um exemplo de tarefa em Processamento de Linguagem Natural que envolve esse tipo de informação é a Modelagem de Linguagem, que corresponde a criar um modelo preditivo da próxima palavra em uma frase [32, 33]. Nesse tipo de tarefa, é fácil perceber que conhecer a sequência de palavras anteriores é importante para a predição. Uma particularidade das RNNs é que esse tipo de rede possui uma memória, que permite registrar que informações foram processadas até o momento atual.




Há muitas maneiras pelas quais as unidades de uma RNN podem estar conectadas entre si. A ilustração à esquerda da Figura 3.9 apresenta a situação mais simples possível, uma RNN que possui apenas uma camada oculta, composta de uma única unidade, denotada por s. Apresentada dessa forma, a rede é aparentemente simples. Entretanto,

repare que a unidade oculta contém uma conexão que a liga consigo mesma. Dessa forma, s recebe sinais que foram gerados por ela própria um passo de tempo no passado. O desdobramento no tempo (time unfolding) de uma RNN corresponde à situação em que essa rede recebe uma sequência de sinais da entrada, um em cada passo de tempo. Na ilustração da direita da Figura 3.9, é apresentada a mesma rede da esquerda desdobrada (unfolded) no tempo. Note que, vista dessa forma, uma RNN que recebe uma sequência de n vetores de entrada pode ser vista como uma rede alimentada adiante (Seção 3.2.1) de n camadas.

Repare também que, embora na Figura 3.9 sejam apresentadas entradas e saídas em cada passo de tempo, dependendo da tarefa específica para qual a RNN deve ser projetada, alguns dos passos de tempo podem não conter nem entradas nem saídas. Isso permite que RNNs aprendam mapeamentos entre sequências de entrada e de saída com diferentes tamanhos: um-para-n (e.g., produzir a legenda correspondente a uma imagem), n-para-um (e.g., produzir a carga de sentimento, positiva ou, negativa, de uma frase de entrada), n-para-m (e.g., traduzir uma frase do inglês para o português). Considerando ainda a Figura 3.9, os vários componentes de uma RNN representados nessa segunda ilustração são descritos a seguir.

• xt é o vetor que representa a entrada fornecida à RNN no passo de tempo t. Por exemplo, se a RNN processa sequências de palavras, então x0 é a representação da primeira palavra, x1 representa a segunda palavra, e assim por diante.

• st é o estado (i.e., o valor de ativação) da unidade oculta no passo de tempo t. Essa unidade representa a memória da rede, porque sua finalidade é registrar que informação foi processada desde alguns passos de tempo no passado até o presente momento. A ativação dessa unidade é computada como uma função recursiva, que depende do estado anterior e da entrada no passo atual: st = σ(Uxt +W st−1 +bs). σ é a função de ativação escolhida (Seção 3.2.1). Repare que s−1, que corresponde ao valor da base da recursão, é normalmente definido como o vetor nulo.

• ot é a saída no passo de tempo t. Por exemplo, se a tarefa corresponde a predizer a próxima palavra em uma frase, então ot é um vetor de probabilidades sobre todas as palavras do vocabulário, i.e., ot = softmax(V st +bo).

• As matrizes U (conexões entre estados ocultos), V (conexões dos estados ocultos para a saída) e W (conexões da entrada para os estados ocultas) são os parâmetros que a rede deve aprender durante o treinamento. De forma similar ao que acontece com as redes convolucionais (Seção 3.4), esses parâmetros são compartilhados por todos os passos de tempo. Esse compartilhamento permite que a rede processe entradas de tamanho variável.

Uma vez definida a arquitetura da RNN, é possível iniciar o treinamento por meio da minimização de uma função de custo J. Nesse caso, uma variante do algoritmo de retropropagação do erro (Seção 3.2.2) é utilizada, a denominada retropropagação através do tempo (Backpropagation Through Time, BPTT). O BPTT pode ser entendido se considerarmos que uma RNN corresponde a uma rede alimentada adiante com uma camada para cada passo de tempo e na qual a matriz de pesos de cada camada é a mesma, i.e., se considerarmos o desdobramento da rede através do tempo. (Aliás, é dessa perspectiva que se originou o nome do algoritmo BPTT.) Dessa forma, após o fornecimento da sequência de entrada, os gradientes são calculados (da mesma maneira que nas redes alimentadas adiante) e posteriormente alterados para manter a restrição de igualdade da matriz de pesos em cada camada oculta. Por exemplo, se w1 e w2 são os valores de conexões correspondentes em duas camadas ocultas distintas (i.e., em dois passos de tempo distintos), então w1 = w2. Para fazer com que esses pesos continuem sendo iguais no próximo passo de tempo, o BPTT calcula ∂J ∂w1 e ∂J ∂w2 e usa a média simples (ou a soma) dessas quantidades para atualizar tanto w1 quanto w2.

### Aplicações

Devido a sua flexibilidade no que concerne ao processamento de entradas de tamanho variável, RNNs têm sido aplicadas recentemente em uma variedade de problemas. De fato, a maioria das aplicações recentes de RNNs envolve o uso de um tipo mais complexo dessa classe de redes proposto em 1997, as denominadas redes LSTM (Long Short-Term Memory) [34]. Basicamente, uma rede LSTM é formada de blocos (ou células) que, por possuírem uma estrutura interna, podem armazenar um valor por uma quantidade arbitrária de passos de tempo. As redes LSTM e sua contrapartida mais moderna (e mais simples), as redes GRU (Gated Recurrent Unit) [35], são especialmente apropriadas para evitar o problema da dissipação dos gradientes (Seção 3.6).

Algumas aplicações das RNNs são reconhecimento de fala [36, 37], geração de textos [38], geração de textos manuscritos (handwriting generation) [39], tradução automática [40] e predição em vídeo [41]. Visto que RNNs são máquinas de Turing completas, em teoria, elas podem realizar qualquer computação. Nesse contexto, tem surgido recentemente aplicações de RNNs baseadas nos modelos de atenção (atention models). No contexto de uma tarefa de visão computacional, um modelo de atenção permite que essa rede direcione sequencialmente seu foco para um subconjunto da entrada por vez. Exemplos de aplicações

desses modelos de atenção são [42] (para geração de imagens) e geração legendas para imagens [43].

## Técnicas para Treinamento de Redes Profundas

Apesar das vantagens computacionais e de ser um modelo mais aproximado do cérebro, o treinamento de redes neurais profundas era uma tarefa de difícil execução até uma década atrás, devido a dois complicadores principais. O primeiro deles é conhecido como a dissipação dos gradientes (vanishing gradients) [44], que é causado por unidades que estão saturadas, conforme descrição a seguir. Durante a aplicação do algoritmo de retropropagação, o gradiente da ativação de cada unidade oculta é calculado e utilizado como um termo multiplicativo para atualização dos pesos correspondentes. Entretanto se a unidade em questão está próxima da saturação, então a derivada parcial resultante será um valor próximo de zero. Isso faz com que o gradiente com relação às pré-ativações seja também próximo de zero, porque seu cálculo envolve multiplicar pela derivada parcial. Isso significa que o gradiente retropropagado para as camadas anteriores será cada vez mais próximo de zero, de tal forma que quanto mais próximo à camada de entrada, maior a quantidade de gradientes próximos de zero. Como resultado, à medida que o gradiente do erro é retropropagado seu valor decresce exponencialmente, de tal forma que o aprendizado nas camadas mais próximas à entrada se torna muito lento. Esse problema afeta não apenas o aprendizado em redes profundas alimentadas adiante (Seção 3.2.1), mas também em redes recorrentes (Seção 3.5). O segundo complicador é consequência do número maior de parâmetros para adequar (os parâmetros de uma rede neural são seus pesos, e quanto mais camadas, mais pesos). De fato, quanto maior a quantidade de parâmetros a ajustar em um modelo, maior o risco de sobreajuste (Seção 3.2.2).

O aprendizado de modelos por meio do treinamento de redes neurais profundas tem sido tradicionalmente tratado como um processo empírico. De fato, soluções para os problemas mencionados acima envolvem o uso de diversos procedimentos que foram surgindo ao longo dos anos e que em conjunto passaram a contar como fatores importantes no sucesso da aprendizagem profunda (além do aumento do poder computacional, da disponibilidade de grandes conjuntos de dados e de melhores arquiteturas de rede). Nessa Seção, descrevemos procedimentos cujo propósito é treinar redes neurais profundas de forma adequada, seja diminuindo o tempo de treinamento, seja diminuindo o erro de generalização dos modelos resultantes.

### Pré-treinamento Não Supervisionado

A ideia subjacente ao pré-treinamento não supervisionado (unsupervised pre-training) é iniciar os pesos de uma rede por meio de uma abordagem não supervisionada [45, 46, 21]. Esse procedimento permite posicionar os pesos da rede em uma região do espaço de parâmetros tal que seja mais fácil para métodos de otimização baseados em gradientes encontrarem um valor ótimo para o objetivo durante o treinamento supervisionado [22]. Durante a aplicação desse procedimento, camadas intermediárias são adicionadas à rede iterativamente. Em cada iteração, uma nova camada intermediária é adicionada à rede. A nova camada intermediária adicionada deve capturar regularidades nos padrões de ativação das unidades contidas na camada anterior. Esse procedimento pode ser aplicado para a construção de redes com um número arbitrário de camadas intermediárias.

Após a fase de pré-treinamento não supervisionado (que substitui a iniciação aleatória dos pesos), a camada de saída (i.e., a que produz uma distribuição de probabilidades sobre os rótulos da tarefa de classificação em questão) é então adicionada à rede. (Os pesos que conectam essa camada com a camada anterior podem ser iniciados de forma aleatória.) Finalmente, a rede completa é treinada da forma supervisionada usual (i.e., com sequências de propagações de sinais adiante intercaladas por retropropagações do erro). Essa segunda fase é denominada sintonia fina (fine-tuning). Uma vantagem do pré-treinamento é que é possível tirar proveito da existência de dados não rotulados, que em geral são mais comuns do que dados rotulados. Outra vantagem é que normalmente o aprendizado com pré-treinamento converge mais rapidamente. nos trabalhos que propuseram originalmente o uso de pré-treinamento [45, 46], os autores realizaram o treinamento não supervisionado por meio de máquinas restrita de Boltzmann (restricted Boltzmann machines). Posteriormente, outros autores propuseram usar pré-treinamento por meio de redes autocodificadoras [47, 21] (Seção 3.3).

### Uso de Unidades Lineares Retificadas

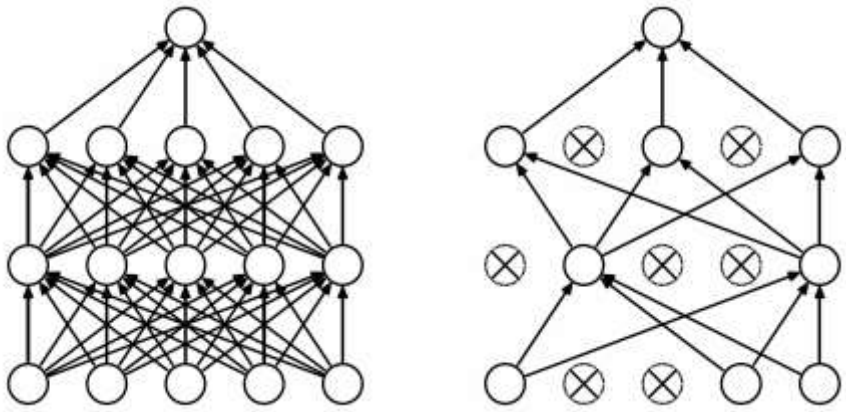
Durante anos, foi um consenso na comunidade de redes neurais usar funções sigmoides (Seção 3.2.1) para implementar a ativação de unidades ocultas, de modo a induzir não linearidades. Entretanto, esse consenso foi questionado em 2010 por Nair e Hinton [48], que propuseram o uso de uma alternativa mais simples, a função de ativação retificada linear (Eq. 5). Neurônios que usam essa função de ativação são chamados de unidades retificadoras lineares (rectified linear units, ReLU). Foi demonstrado por meio de experimentos [28] que em geral a função ReLU produz um erro de treinamento menor, por conta de não ser tão suscetível ao problema da dissipação dos gradientes quanto as funções sigmoide e tangente hiperbólica. Além disso, essa função de ativação reduz o tempo de convergência dos parâmetros (por conta de sua computação não envolver exponenciações).

Por exemplo, em [28], os autores apresentam uma comparação entre as curvas de convergência em uma rede de convolução com 4 camadas em duas variações, uma com neurônios ReLU e outra com neurônios tanh no conjunto de dados CIFAR-1013. Nos experimentos apresentados, a primeira variação converge 6 vezes mais rápido.

Em 2011, o grupo de Yoshua Bengio [49] demonstrou que ReLUs permitem o treinamento supervisionado de redes profundas sem a necessidade de utilizar pré-treinamento não supervisionado (Seção 3.6.1). Em 2015, uma rede convolucional treinada no conjunto de dados ImageNet com a função ReLU pela primeira vez atingiu precisão de classificação maior do que a alcançada por um ser humano [50].

### Desligamento (Dropout)

Essa é outra técnica também aplicável ao treinamento de uma rede neural para reduzir o risco de sobreajuste [51, 52]. Na fase de propagação adiante (forward propagation), antes de um padrão ser apresentado à camada intermediária de índice k, cada entrada de um vetor binário m(k) é gerado por meio de uma distribuição uniforme com parâmetro p (p = 0,5 é um valor usual). Esse vetor é então usado como uma máscara para desativar algumas das unidades da camada k. Mais especificamente, a pré-ativação da camada k ocorre de forma usual (conforme a Eq. 1), mas a sua ativação é modificada ligeiramente: h(x) = g(a (k) (x))· m(k) (compare com a Eq. 2). Efetivamente, essa modificação faz com que a ativação das unidades correspondentes a valores iguais a zero na máscara m(k) seja também igual a zero [53]. A Figura 3.10 o efeito da aplicação do desligamento: a imagem da esquerda é a configuração original da rede, e a da direita corresponde à configuração após o desligamento de algumas unidades.



A aplicação de desligamento requer a definição de um hiperparâmetro, a probabilidade p que controla a intensidade de desligamento. Se p = 1 implica nenhum desligamento, e quanto menor o valor de p, maior a taxa de desligamento. Valores típicos de p para as

unidades ocultas estão na faixa de 0,5 a 0,8 [52]. De acordo com [54], o procedimento de desligamento pode ser interpretado (e mesmo justificado) como uma forma adaptativa de acréscimo de dados (data augmentation) [55]. De fato, esses autores apontam que definir a mudança de ativação de algumas unidades ocultas para 0 é equivalente a fornecer na entrada da rede um exemplo moldado para produzir ativações iguais a 0 para aquelas unidades, ao mesmo tempo em que se mantém a mesma ativação para todas as demais unidades daquela camada. Uma vez que (1) cada exemplo moldado é muito provavelmente diferente do exemplo original correspondente (por conta da aplicação da máscara estocástica), e que (2) cada máscara diferente corresponde a um exemplo moldado de forma diferente, então é intuitivo concluir que o procedimento de desligamento produz resultado semelhante a aumentar a quantidade de exemplos de treinamento.

### Normalização em Lote

Uma transformação usual durante o treinamento de uma RNA é normalizar o conjunto de dados de treinamento de acordo com a distribuição normal padrão (i.e., média igual a zero e variância igual a 1) para evitar problemas de comparação devido às diferentes escalas usadas nos dados. Ocorre que durante a passagem dos exemplos normalizados através das camadas da rede, esses valores são novamente transformados (por meio das pré-ativações e ativações), podendo fazer com que os dados de entrada de algumas camadas ocultas fiquem desnormalizados novamente. Esse problema, conhecido como mudança de variável interna (internal covariate shift), é tanto mais grave quanto mais profunda for a rede a ser treinada. Esse problema aumenta o tempo de treinamento porque implica na definição de uma taxa de aprendizagem pequena, além de propiciar a dissipação dos gradientes. A Normalização em Lote (Batch Normalization) [56] é um mecanismo proposto recentemente para resolver o problema acima, e que consiste em normalizar os dados fornecidos a cada camada oculta. A normalização é aplicada em cada mini-lote. 14, para aumentar a eficiência durante a aplicação da transformação.

De acordo com os experimentos realizados pelos autores da técnica [56], ela também produz um efeito de regularização sobre o treinamento, em alguns casos eliminando a necessidade de aplicar o desligamento (Seção 3.6.3). Uma aceleração significativa do tempo de treinamento também foi observada, resultante da diminuição de 14 vezes na quantidade de passos de treinamento necessários, quando comparada ao tempo de treinamento sem o uso da normalização.

## Estudo de Frameworks

De acordo com Gokula Krishnan Santhanam, de RTH Zurich, existem quatro requisitos fundamentais para as ferramentas de AP:

• possibilidade de trabalhar com tensores 1 (o que implica a existência de um objeto tensorial sobre o qual é possível realizar operações, isto do ponto de vista de uma linguagem orientada por objetos, como é o caso do Python amplamente utilizado neste domínio) .

• capacidade para computação gráfica e de otimização • ferramentas de auto-diferenciação .

• existência de extensões que permitam trabalhar com BLAS/cuBLAS2 (implementação subrotinas de álgebra linear elementar que é executada nos processadores gráficos - GPUs) e cuDNN3 (bibliotecas que permitem a implementação de redes neuronais profundas que são executadas nos processadores gráficos).

Estes permitirão tornar a Framework numa ferramenta útil e completa. Outros fatores não referidos pela autora, mas que são essenciais para a sua utilização têm a ver com a facilidade de instalação, configuração, implementação e a usabilidade, bem como a velocidade de execução do código. Estes parâmetros vão ser estudados para as três frameworks consideradas, de todo o espectro disponível. A próxima secção introduz frameworks para AP, ferramentas importantes utilizadas neste trabalho e plataformas para grandes quantidades de informação.

## Frameworks, ferramentas e plataformas

A motivação que esteve na origem da escolha das frameworks deve-se à representatividade que elas possuem, com é o caso de Theano e do TensorFlow . A terceira framework foi considerada principalmente por ser extremamente recente e ter potencialidades para ser uma das mais representativas num futuro próximo, a Microsoft CNTK.

Existem, no entanto, outras frameworks tais como:

• OpenCV - embora já exista desde 1999, começa a mostrar capacidade de entrar nesta área, a partir de 2010, principalmente com as suas valências ao nível de aplicações móveis e multimédia, por exemplo associar a realidade aumentada, ao IoT e ao AP;

• Torch - utiliza como base a linguagem Lua que está pouco divulgada;

• RStudio - é uma interface gráfica para uso da linguagem R , permitindo a integração de diversos componentes;

• Matlab - possui o Deep Learning Toolbox e é software proprietário;

• Caffe - a sua implementação é difícil, bem como a instalação que parece estar repleta de “bugs”, é um sistema pouco versátil;

• Lasagne - consiste numa biblioteca de python implementada numa nova camada do Theano, mas com a qual não é muito fácil a implementação. Depois do estudo realizado, nesta tese, torna-se claro que quando comparada com o Keras parece que o código deste é mais simples e claro;

• Keras citeKeras - consiste numa biblioteca de python implementada como uma nova camada do Theano e do TensorFlow. Possui um código claro e transparente. Por defeito esta usa o TensorFlow em segundo plano;

• MxNet - é uma framework de AP que permite definir, treinar e implementar redes neuronais profundas para diversos dispositivos, desde os móveis até à Cloud;

• Deep Learning 4j - utiliza a linguagem Java, podendo ser útil, por exemplo, na área das aplicações móveis desenvolvidas para Android. Nomeadamente, naquelas que exigem analise de dados em tempo real, como por exemplo aplicações de realidade aumentada. No entanto, são precisas muitas linhas de código para a implementação, devido à estrutura da própria linguagem. Utiliza o Maven, também conhecida por Apache Maven, e, é uma ferramenta de automação de compilação utilizada primariamente em projetos Java;

• SAS - é uma plataforma vastíssima na qual começa a emergir a AP dentro deste software proprietário;

• Intel Deep Learning SDK - utiliza o Caffe como base. Facilita a implementação dado que esta é feita através de uma interface web. Mas devido a usar o Caffe tem muitas limitações;

• Chainer- apesar de não ser objeto de estudo, é importante falar dele porque é a única framework que permite ajustar o código enquanto ele está a ser executado, o que permite

poupar tempo na execução de projetos nesta área, pois não é necessário que um processo termine para o avaliar;

• DLib-ML - é uma biblioteca de C++ que tem um Toolkit para ML e Python inclusive. Mas é a componente de C++ que está amplamente desenvolvida, pois a de Python fica-se pela deteção de objectos em imagens e por SVM 4.

Em 2015, também a Apple resolveu apostar no AP para todos os seus dispositivos e criou o DeepLearningKit que pode ser instalado em qualquer um dos dispositivos que comercializa. O DeepLearningKit vem possibilitar a integração de algoritmos de AP nos sistemas operativos da Apple. Consegue importar modelos de outras frameworks, é uma biblioteca que permite utilizar a capacidade de processamento das placas gráficas e implementar modelos de AP para as aplicações da Application Store.

O gráfico apresentado na figura 3.1 evidência que nos últimos 2 anos ocorreu um “boom” produtivo na área da AP com o surgimento de novas frameworks. Constata-se que a linguagem de programação mais utilizada e que por isso mesmo é comum a praticamente todas as frameworks é o Python, seguida do C++ e depois algumas mais específicas como a Lua, BrainScript, Go, etc.

Para além das ferramentas referidas existem outras que têm a sua aplicação na área da aprendizagem automática. Aqui fica um breve sumário de algumas delas :

• Scikit-learn - consiste numa nova camada de abstração de Python que trabalha em cima do Numpy, Scipy e Matplotlib.

• Shogun - é uma biblioteca de AA com capacidade para trabalhar com várias linguagens: C++, Java, Python, C#, Ruby, R, Lua, Octave e Matlab.

• Accord Framework/AForge.net - permite a implementação de algoritmos de AA em .net.

• Mahout - é uma framework de Hadoop5 focada em aplicações para AA e Mineração de Dados;

• MLlib - é uma biblioteca do Apache para ser usada no Spark e no Hadoop. Tal como a Mahout também está direccionada para aplicações de AA, embora seja muito mais eficiente pois demora cerca de 10x a 100x menos a processar a informação;

• H2O - fortemente orientada para o Hadoop trabalha com Python, Scala e R e é mais usada no domínio financeiro (detecção de fraudes), fazendo inclusive previsões.

• Cloudera Oryx - desenhado a pensar no Hadoop, permite trabalhar com dados em tempo real, o que pode ser bastante útil para detecção e combate ao SPAM, entre outras aplicações. • GoLearn - A linguagem usada é o Go (linguagem da Google), uma linguagem com apenas 5 anos e que se pauta pela simplicidade e usabilidade/versatilidade.

• Weka - muito útil no que diz respeito ao Data Mining (Mineração de Dados), pois inclui diversos algoritmos e bibliotecas que permitem analisar conjuntos de dados sem ter que escrever código. A linguagem de base é o java e existem muitas contribuições e imensas ”packages”(tal como acontece no R/RStudio).

• CUDA-Convnet2 - A linguagem de base é o C++ mas as saídas da rede implementadoa pode ser lido no formato Pickle usado pelo Python.

• ConvNetJS - versão das CNN para usar no browser. A linguagem de base é o javascript.

Para além do já referido existem alguns utilitários de extrema importância, tais como:

• Docker - é uma plataforma aberta usada por Administradores de Sistemas e Developers para criar, distribuir e executar aplicações em qualquer lado; funciona de como os ambientes Python6 , mas sendo mais completo, pois permite configurar todo um sistema independentemente da linguagem, da versão e da aplicação. Permite criar sistemas com configurações muito variadas, diferentes versões de compiladores, de sistemas operativos,

etc. É melhor que o Git pois possui um ficheiro de configuração “DockerFile” que contém toda a informação necessária para a criação das imagens e dos componentes designados por ”Containers”quer reproduzem com exatidão as condições em que foram criados. Pode ser considerado um produto ”chave na mão”, que basta ser executado para reproduzir tudo o que foi configurado;

• Git - é uma ferramenta útil para divulgação de código na rede, embora por vezes contenha informação pouco clara, mas que se deve essencialmente à falta de cuidado dos autores;

• Jupyther - é uma ferramenta muito útil para o ensino, para o estudo e para a exploração de novos métodos, novas técnicas. Pois permite a execução de pequenas partes de código de forma iterativa.

• Anaconda - é uma ferramenta que surge para facilitar a instalação de bibliotecas que de outra forma seria mais complexo, ou seja, da forma tradicional, compilando o código a partir da fonte.

Quando se quer passar para um domínio de grande dimensão de informação a ser analisada já se recorrem a outras plataformas, como por exemplo:

• Sistemas Cloud (AWS, Azure, Google) [Ama, Micb, Good]

• Sistemas de Alta Disponibilidade, que funcionam 24/7 sem falhas. Podem ser sistemas de clusters, podem ser sistemas com arquiteturas que permitem redundâncias. Estes sistemas são implementados quer ao nível de hardware, quer ao nível do software. Por exemplo, os clusters usados pela Alfresco7 .

• Big Data (Hadoop, Hive, MapReduce, HDFS, Spark, entre outras). Nesta área o Apache tem um leque diversificado de ferramentas que podem ser usadas com os mais diversos fins. Por exemplo, o Spark é usado para dividir os dados de grande dimensão em pequenas parcelas, e, depois processá-los em paralelo. Em certa medida, já o MapReduce permite reduzir a dimensão dos dados original. O Apache Hadoop, permite a escalabilidade e o HDF5, que permite criar uma organização por contentores, permitindo também o acesso aos dados. Por seu lado, o Apache Hive é um Data Warehouse. Percebe-se que o Apache tem muitas ferramentas que podem ser utilizadas por quem trabalha com dados de grandes dimensões.

## Frameworks Estudadas

Dada a grande quantidade de frameworks disponíveis decidiu-se estudar e fazer uma análise detalhada de um subconjunto daquelas apresentadas. As frameworks escolhidas foram Theano, TensorFlow e CNTK. Nas sub-secções seguintes introduzem-se estas frameworks (e ainda a biblioteca Keras), faz-se uma análise comparativa da sua utilização segundo o número de publicações e problemas encontrados pelos utilizadores na utilização destas e na implementação do código desenvolvido por eles e apresenta-se uma avaliação pessoal sobre a qualidade da documentação, facilidade de instalação e implementação das frameworks escolhidas.

### Theano

O Theano é uma biblioteca de Python que permite definir, otimizar e avaliar expressões matemáticas, ou funções, com recurso a vetores, matrizes e sistemas multi-dimensionais de forma eficiente (Tensores). As suas características permitem a integração com outras bibliotecas como por exemplo a Numpy , a configuração para que o código seja processado, quer nos CPU's 9 , quer nos GPU's 10. Os cálculos podem ser cerca de 140x mais rápidos quando processados nos GPU's, considerando que se usa float32 . A framework consegue ainda diferenciar código simbólico, permitindo derivar funções com várias variáveis, ou seja, com várias entradas, o que se torna verdadeiramente útil para a sua aplicação nas redes neuronais. Esta é talvez a razão que a faz ser uma das frameworks mais utilizadas.

As otimizações são rápidas e estáveis. Por exemplo consegue resolver a função log(1 + x) para valores de x muito pequenos. Por outro lado, consegue obter melhores desempenhos quando comparado com o processamento realizado em Matlab ou em C++ . Como o Theano usa como linguagem o Python, consegue gerar código C de forma dinâmica, o que permite a avaliação das expressões mais rapidamente. O Theano tem sido largamente utilizado na investigação científica desde 2007, e, como utiliza a linguagem Python é suficientemente simples para ser usado ao nível do ensino universitário. Um bom exemplo disso são as aulas de "deep learning/machine learning" da Universidade de Montreal.

Alguns dos modelos disponíveis nesta ferramenta são:

• Dropout

• Rede convoluída do Alex

• RNN CTC

• TreeLSTM

• Variational Recurrent Neural Networks

## TensorFlow

O TensorFlow é uma biblioteca de Python desenvolvida pelos investigadores da Google que trabalhavam no projeto Google Brain e tem como homónimo o Theano. Apesar disso existe uma diferença significativa entre este e o Theano, que consiste na possibilidade do TensorFlow usar uma biblioteca de “Data Flow Graph” para computação numérica. Esta ferramenta foi criada com o intuito de facilitar a vida a que trabalha com a AA, embora possa ser utilizada noutras áreas, dada a sua versatilidade. Devido à sua arquitetura permite com uma simples API utilizar os mais variados recursos de hardware, sejam eles CPU’s ou GPU’s de computadores, de servidores ou de dispositivos móveis. O TensorFlow só suporta processamento no GPU a 32 bits, ao contrário do Theano que permite o uso quer dos 32 bits, quer dos 64 bits. Quem usa o TensorFlow pode usar uma funcionalidade designada por TensorBoard, na qual é possível através de um endereço web testar o código e monitorizar a sua execução. O TensorFlow usa o GPU por defeito. Mas existe uma vantagem enorme nesta framework que é a capacidade de utilizar recursos disponíveis via rede computacional.

Os modelos disponíveis nesta ferramenta são:

• autoencoder - diversos tipos de autoenconders.

• differential - modelos para respeitar a privacidade dos estudantes relativamente a professores diferentes.

• im2txt - rede neuronal para captar imagens e convertê-las para texto.

• inception - CNN profunda usada em visão computacional.

• namignizer - reconhecer e gera nomes.

• neural - permite o processamento das redes nas gráficas com possibilidade de paralelismo.

• neural - rede neuronal aumentada através de recurso à lógica e a operações matemáticas.

• resnet - composto por redes neuronais residuais profundas e alargadas.

• slim - classifica imagens usando modelos TF-Slim.

• swivel - aplicação do algoritmo Swivel para gerar palavras embebidas.

• syntaxnet - contêm modelos neuronais para sintaxe em linguagem natural.

• textsum - usa uma abordagem sequencial de sequência a sequência, permitindo um resumo textual.

• transformer - rede para transformação espacial, permitindo a manipulação espacial dos dados dentro da rede.

Estes modelos foram desenvolvidos por diversos autores e são mantidos pelos mesmos .

### Keras

A Keras é uma biblioteca de python que pode ser usada numa segunda camada de abstração do Theano ou do TensorFlow. A sua grande vantagem é permitir escolher uma destas frameworks para ser executada no “backend” e facilitar o processo de implementação do código. Existem dois tipos de modelos no KERAS:

• sequencial - é um conjunto de camadas compactadas e organizadas de forma linear, ou seja, uma seguidas às outras.

• API funcional - permite com os dados de entrada e de saída da rede construir um modelo que seja representativo desses mesmos dados.

Sendo esta biblioteca facilitadora da implementação do código, optou-se por usá-la em conjunção com o Theano e com o TensorFlow, no processo de análise e comparação.

### Microsoft CNTK

A Microsoft CNTK é um componente de software que torna fácil o desenho e o teste de redes computacionais, como as redes neuronais profundas . Tal como os concorrentes, é utilizado para treinar e testar diversos tipos de redes neuronais. A configuração da rede é centralizada num ficheiro de texto. Tal como o Theano e o TensorFlow, também permite processamento paralelo e direcionamento do mesmo para os CPU’s e/ou GPU’s, suportando o sistema CUDA . Pode ser utilizado com uma linguagem própria, a BrainScript , ou em alternativa, com Python ou C++. Os modelos disponíveis nesta ferramenta são .

• AlexNet - CNN desenvolvida por Alex Krizhevsky e que venceu o ILSVRC2012 num processo de classificação.

• AN Speech (DNN e LSTM) - modelos usados na área do discurso e em que a sua estrutura de base consiste numa rede neuronal artificial.

• ConvNet - rede neuronal convolucional desenvolvida na Universidade de Standford.

• VGG11

• AE - codificação automática, em inglês designa-se por Auto-Encoder. • Aprendizagem por reforço.

O dinamismo na implementação de modelos nesta nova framework é bem evidente. Das diversas vezes que o site da Microsoft foi consultado, o incremento de novos modelos foi significativo.

## Trabalho Desenvolvido

Para a comparação entreframeworks, seguiu-se o seguinte procedimento:

1. procedeu-se à pesquisa das frameworks existentes.

2. entre elas selecionaram-se três: Theano + Keras, TensorFlow + Keras e Microsoft CNTK.

3. definiram-se os critérios de avaliação e as escalas de classificação para: instalação, implementação e documentação.

4. escolheram-se três modelos do tipo CNN para análise do comportamento das três frameworks seleccionadas, para um estudo mais profundo, com dados supervisionados.

5. avaliou-se o género de recursos utilizados pelas três frameworks em estudo; neste caso a RAM do GPU usada, visto os cálculos terem sido processados no GPU.

6. quantificou-se o número de linhas de código necessárias para a implementação de cada um dos modelos utilizados.

7. registou-se o tempo de treino para cada modelo e para cada framework.

8. calculou-se a exatidão, quando se altera a dimensão de dados de treino para o modelo mais elaborado, usando a framework que demonstrou ter melhor pontuação nos itens anteriormente classificados.

## Resultados e Discussão

Colocando num gráfico as frameworks em função do número de publicações (sejam elas, artigos, livros, vídeos, cursos, código disponibilizado no GitHub, ou mesmo imagens de sistemas disponibilizado no DockerHub) verifica-se que as que aparentemente têm mais

representatividade a nível global são o RStudio e o DL4J, talvez por o primeiro ser frequentemente utilizado por Cientistas de Dados 12 e Analistas de Dados 13 no domínio do Big Data e do IoT14 A utilização do DL4J reflete-se pelo facto de a linguagem JAVA ser uma das mais divulgadas e utilizadas (ver gráfico da figura).

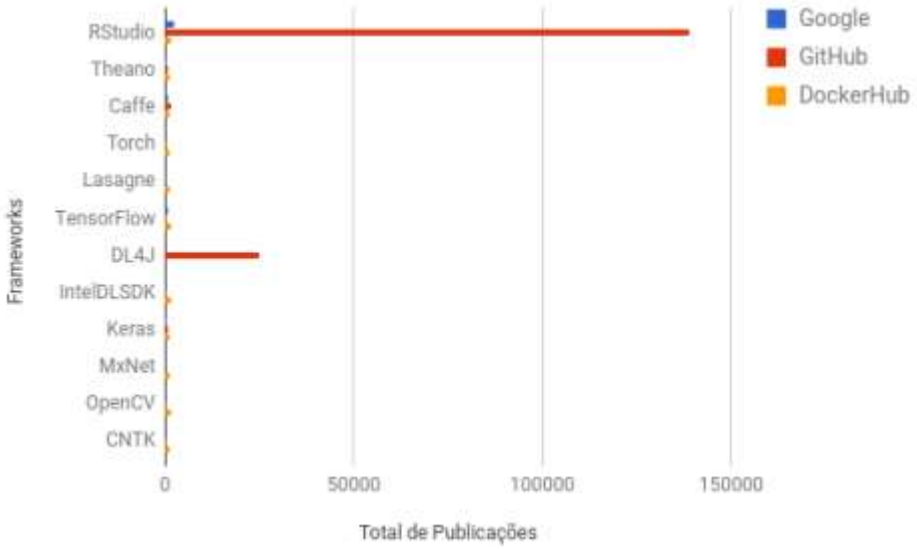



Figura 3.2: Relação entre as frameworks e o número total de publicações na Google, GitHub e DockerHub.

Dada a grande disparidade, para o RStudio e para o DL4J em relação ao GitHub, principalmente ao nível dos valores, resolveu-se retirar estas duas frameworks e analisar a relação entre as restantes. Obteve-se o gráfico apresentado na figura 3.3.

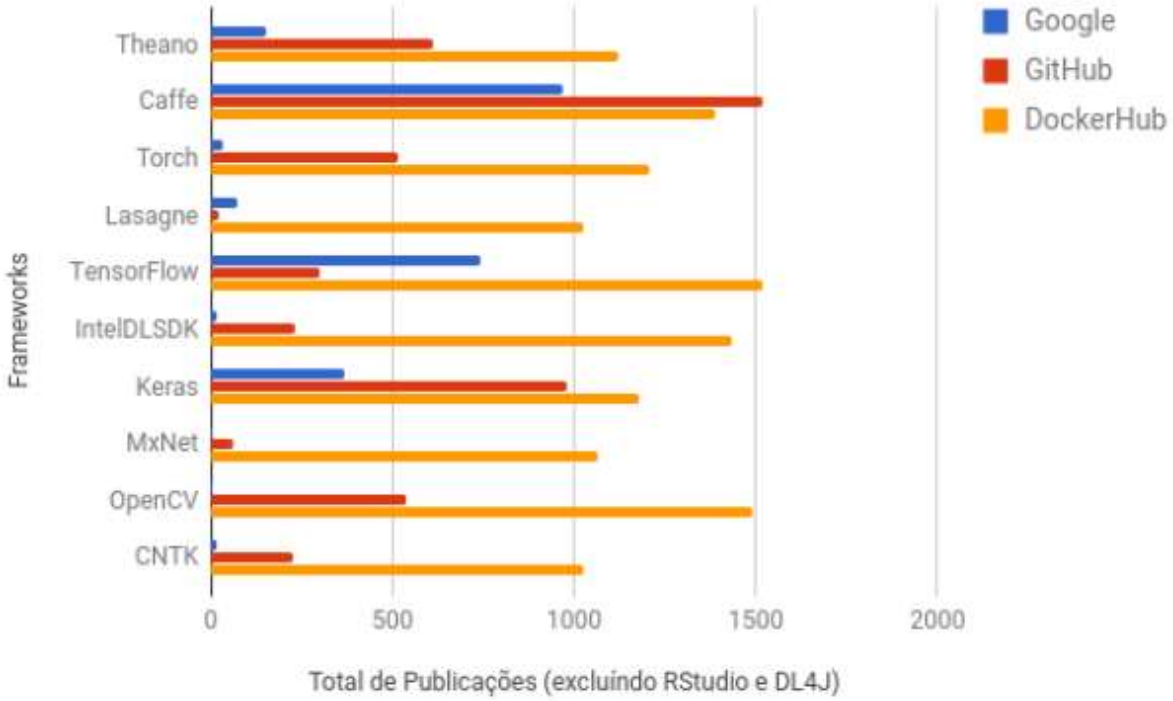


Figura 3.3: Relação entre as frameworks e o número total de publicações após exclusão do RStudio e do DL4J.

Da análise do gráfico da figura 3.3, infere-se que existe uma clara tendência para o DockerHub passar a ser uma ferramenta utilizada para o desenvolvimento e teste de novos sistemas. Este resultado compreende-se pelo facto de o DockerHub ser utilizável em qualquer Sistema Operativo, em qualquer rede, inclusive na Cloud e permitir construir diversas configurações que ficam automaticamente disponíveis para quem as quiser utilizar. Mas como neste trabalho o objetivo é estudar apenas três frameworks, vamos ver o que acontece quando se retiram todas as outras (ver gráfico da figura 3.4).

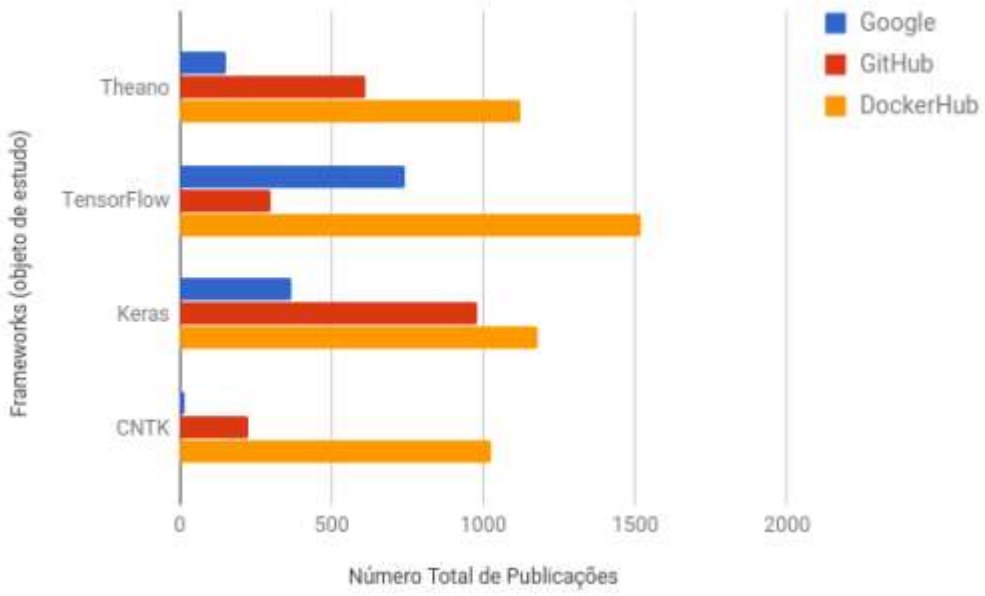



Figura 3.4: Comparação entre as três frameworks em análise e o número total de publicações na Google, no GitHub e no DockerHub.

Também aqui se evidencia a preponderância do Docker face aos outros sistemas. Estes dados estatísticos foram obtidos através da pesquisa “[frameworkname] deep learning” em cada um dos motores de pesquisa disponibilizados por cada um dos sistemas.

Vejamos agora outro tipo de análise que também pode ser interessante. Vamos comparar as frameworks com os problemas encontrados (abertos no GitHub, fechados no GitHub e no Stackoverlfow).

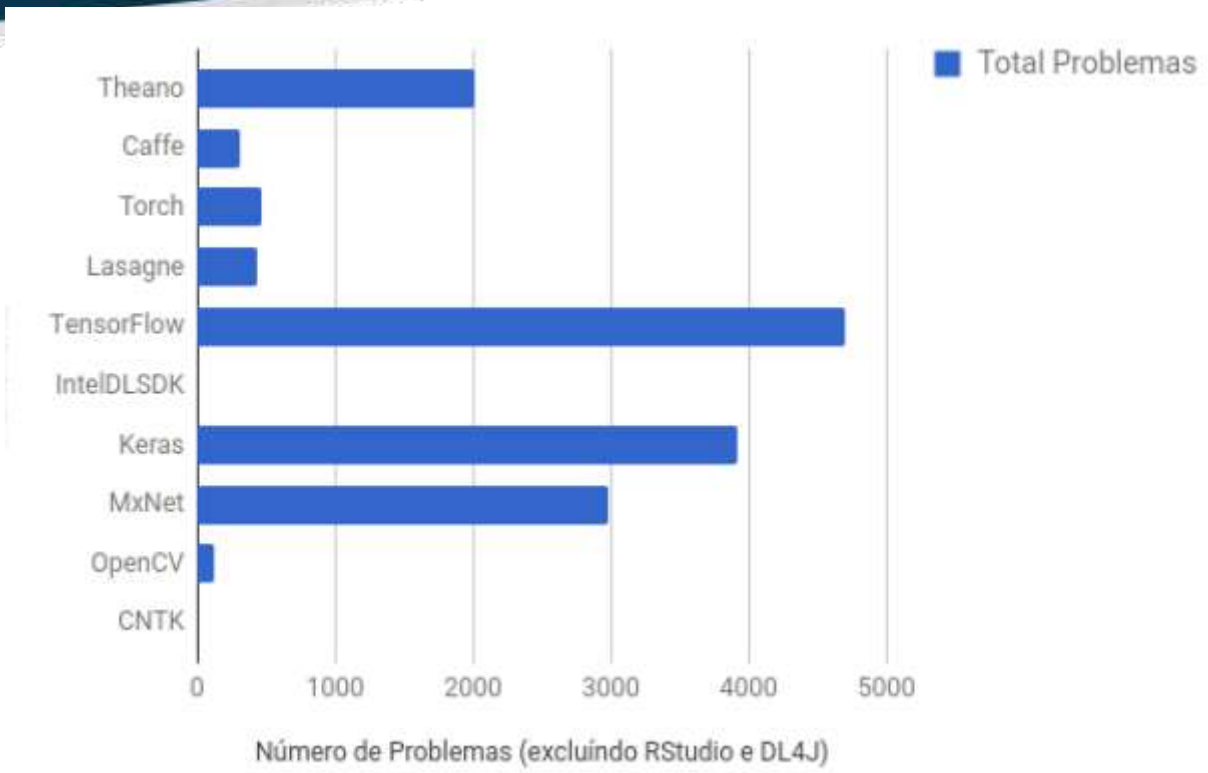



Figura 3.5: Comparação entre as frameworks e o número total de problemas encontrados.

Ao observarmos o gráfico da figura 3.5 temos uma noção de que o RStudio não apresenta dados nestes sistemas talvez por ser um produto “chave-na-mão”, ou, por usar outro lugar para discutir e apresentar os problemas encontrados, como por exemplo um fórum específico para R. Quanto ao IntelDLSDK e ao CNTK, como são muito recentes também ainda não são amplamente usados e testados como as outras.

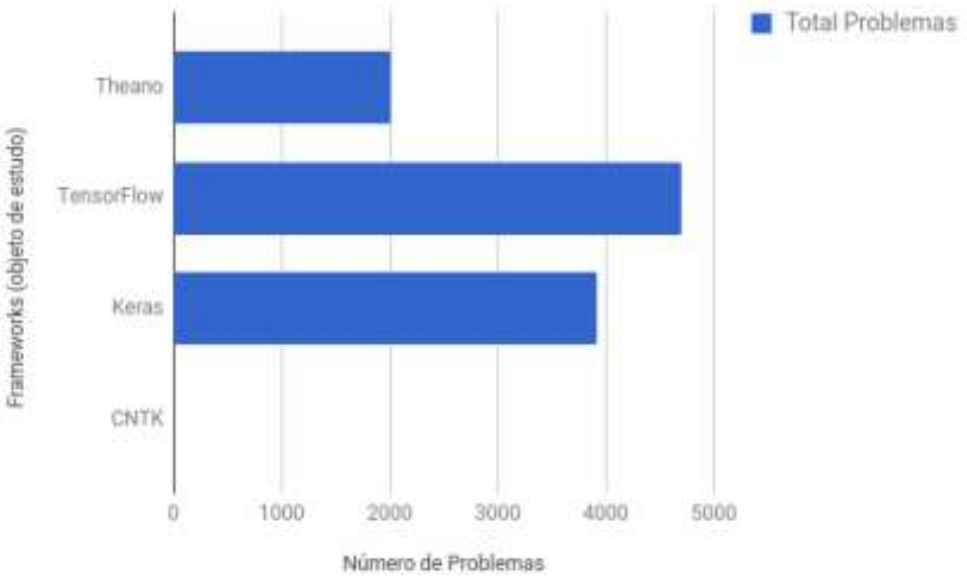


Figura 3.6: Relaçao entre as frameworks e o numero total de problemas encontrados.

Excluindo as outras frameworks, centrando a atenção exclusivamente nas selecionadas, obtêm-se o gráfico da figura 3.6.

Curiosamente constata-se que o Theano parece ter menos dissabores no que diz respeito a bug's e outro tipo de problemas que podem surgir quando se tenta instalar e quando se tenta implementar o código. Parece por isso mais maduro. O CNTK não permite grandes conclusões, pois é demasiado recente, não se tendo dados que possam servir para a análise. Os dados estatísticos deste estudo foram obtidos a 11-02-2017.

A fase a seguir da comparação passa por avaliar percentualmente as diversas frameworks no que diz respeito aos seguintes parâmetros: Documentação, Instalação e Implementação. Tendo sido definida uma escala para cada um deles apresentada no ponto 4.1.

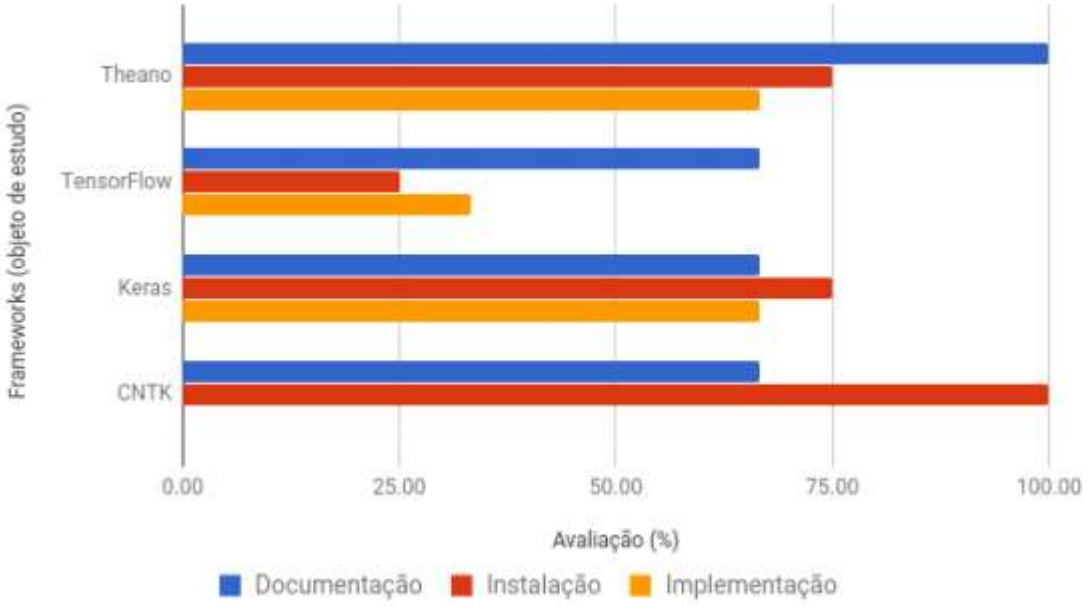



Figura 3.7: Avaliação das frameworks em estudo no que diz respeito à documentação disponibilizada, à instalação e à implementação.

A figura 3.7 retrata a cotação percentual obtida para cada um dos itens avaliados, evidenciando-se a facilidade de instalação da CNTK e a facilidade de implementação do Keras em conjunto com o Theano.

## Estudo de Caso I

## CLASSIFICAÇÃO DE ESPÉCIES FLORESTAIS USANDO APRENDIZAGEM EM PROFUNDIDADE (DEEP LEARNING)

### INTRODUÇÃO

A indústria madeireira lida diariamente com diferentes tipos de madeiras, extraídas de diferentes espécies florestais. A falta de fiscalização apropriada, entretanto, pode fazer com que madeiras de uma determinada espécie sejam comercializadas como se pertencessem a outra espécie. Por exemplo, espécies mais valorizadas podem ser trocadas por espécies de menor valor, o que pode causar prejuízo a um determinado comprador. Mais gravemente, o comércio de madeiras extraídas de espécies cuja comercialização e ilegal pode acarretar em um prejuízo ambiental ao país e ao planeta como um todo. A espécie de uma árvore pode ser, de certo modo, facilmente reconhecida quando a mesma encontra-se em seu habitat natural. Mas depois que a árvore é extraída e cortada em tábuas, muitas vezes apenas um especialista é capaz de afirmar de qual espécie a tábua foi extraída. Embora as autoridades governamentais estejam se esforçando cada vez mais para que haja um maior controle neste tipo de indústria, o controle apropriado da comercialização de madeiras dependeria da contratação e recrutamento de um grande número de especialistas. Como o orçamento dos governos é geralmente limitado, o reconhecimento automático de espécies florestais, realizado por computadores (como ilustrado na Figura 1), consiste em uma alternativa de menor custo para tratar este problema. O desenvolvimento deste tipo de sistema pode ser realizado dados avanços recentes relacionados a Visão Computacional [Forsyth & Ponce, 2002] e o Reconhecimento de Padrões (RP) [Duda & Hart, 2000].

Contudo, estas tarefas geralmente requerem um grande número de especialistas, o que demanda tempo e investimentos. Nesse contexto, o desenvolvimento de um sistema automatizado poderia reduzir os custos e tornar este processo mais rápido e eficaz. Vários sistemas foram propostos na literatura para o reconhecimento de espécies florestais por meio do desenvolvimento de softwares que sejam capazes de reconhecer uma espécie a partir da

imagem capturada de uma madeira, auxiliando o ser humano. Neste estudo, modelos de aprendizagem profunda (Deep Learning) são investigados para melhorar a precisão em diferentes padrões de reconhecimento.

## METODOLOGIA

No tipo de sistema que estudamos no projeto, uma imagem digitalizada de uma madeira é utilizada como entrada e a espécie florestal da tábua é automaticamente informada como saída (Figura 1). Uma das principais dificuldades é a implementação de um método eficiente de extração de características de texturas para tais imagens. Considerando isto, neste trabalho, são realizadas pesquisas que visam apresentar uma investigação de técnicas de aprendizagem profunda.

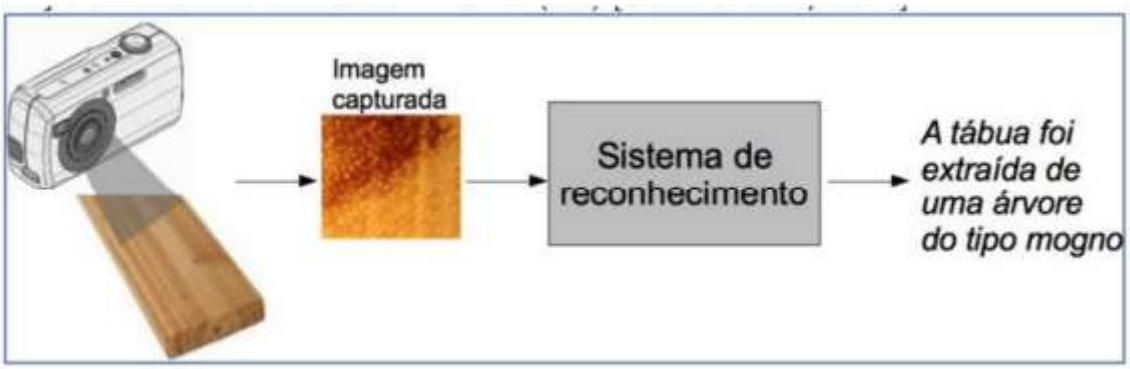



Figura 1: Concepção geral do sistema

Nessas técnicas utilizam-se dados brutos (por exemplo, pixels de imagem) como entrada para os modelos e deixa-se aprenderem representações intermediárias para domínios em que as principais características de formalizar para o reconhecimento. Suas abordagens inovadoras dispensam um pré-processamento e geram automaticamente propriedades invariantes da camada de entrada. Também possuem uma forte capacidade de realizar a classificação usando apenas um fragmento relativamente pequeno de texturas, o que é ideal para o problema de classificação abordado neste estudo. Nesse contexto, será utilizada uma arquitetura de rede neural convolutiva como técnica de Deep Learning. Abaixo descrevemos o método proposto em detalhes e todo o procedimento de implementação, já que rede neural convolutiva é composta por diferentes camadas, tais como:

1) Camadas Convolucional: Esta camada é composta por um conjunto de filtros (ou kernels) treináveis que são aplicados em toda a imagem. Esses filtros são convoluídos (através do produto escalar) com os dados de entradas para obter um mapa de características. O filtro é percorrido por toda a imagem, sendo controlado pelo parâmetro conhecido como stride.

Sobre o resultado da convolução é aplicado a função de ativação - ReLU (unidade de retificação linear), em cada elemento do resultado da convolução.

2) Camadas de Pooling: É utilizada com objetivo de reduzir o tamanho espacial das matrizes resultantes da convolução e capturar pequenas invariâncias com informações mais representativas. Consequentemente, essa técnica reduz a quantidade de parâmetros a serem aprendidos na rede, contribuindo para o controle de sobre ajuste (overfitting).

3) Camada totalmente conectada: É completamente conectada com a camada anterior. Como a camada totalmente conectada vêm após uma camada convolucional ou de pooling, é necessário conectar cada elemento das matrizes de saída de convolução em um neurônio de entrada. Contudo, para entrarem, as matrizes precisam passar por uma alteração em sua estrutura chamado flatten sendo colocados de forma linear para camada totalmente conectada. É nesta camada que o algoritmo de treinamento supervisionado backpropagation é empregado. O erro obtido nesta camada é propagado para que os pesos dos filtros das camadas convolucionais sejam ajustados também. Em seguida, nas unidades de saída, utilizase uma função softmax para se obter a probabilidade de dada uma imagem de entrada pertencer a uma classe na saída.

Uma ilustração das camadas supracitadas é apresentada na Figura 2:

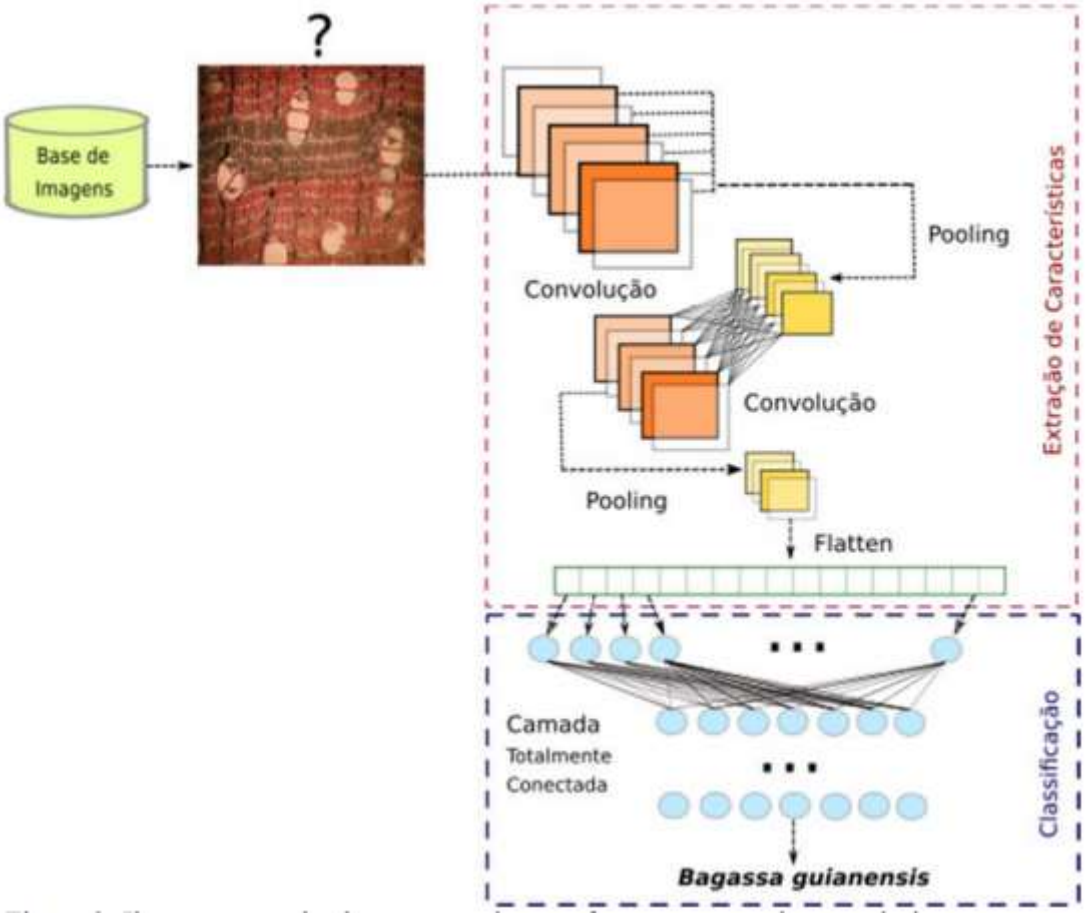


Figura 2: Ilustração geral sobre as camadas que formam uma rede convolutiva.

## FUNDAMENTAÇÃO TEÓRICA

Na literatura, observamos que o desenvolvimento de sistemas de reconhecimento espécies florestais tem atraído o interesse de vários centros de pesquisa nos últimos anos. Entretanto, os requisitos em termos de pré-processos aplicados a madeira antes que esta seja processada computacionalmente faz com que estas abordagens sejam de difícil aplicação na prática. Mas recentemente, em [4] foi proposto um sistema de reconhecimento capaz de reconhecer espécies de madeiras a partir de câmeras de baixo custo, demonstrando que é possível resolver este problema. Técnicas de classificação de textura têm tem sido explorado por vários autores nos últimos anos, assim como: o uso de filtros Gabor e co-ocorrência matrizes (GLCM), Padrões binários (LBP) para extrair recursos relevantes de as imagens, etc. Recentemente, modelos de aprendizagem profunda (Deep Learning) estão sendo utilizados, como o estado da arte em muitos domínios, como revisto por Bengio em [1], Além de melhorar a precisão em diferentes padrões problemas de reconhecimento, essa técnica move o aprendizado de máquina para o descoberta de múltiplos níveis de representação.

## PROTOCOLO EXPERIMENTAL

A base de imagens original de 2240 amostras coloridas foram convertidas em nível de cinza e dividida em 3 conjuntos independentes: treinamento 10 amostras por classe, validação e teste 6 e 4 respectivamente. Para diminuir a complexidade da rede as imagens originais foram reduzidas de 1024x768 para 640x640. As bases de treinamento e validação foram utilizadas para a coleta de resultados e análises dependendo dos diferentes parâmetros aplicados. Ao final, o modelo que apresentou os melhores resultados sobre o conjunto de validação foi utilizada para a obtenção dos resultados finais sobre o conjunto de teste.

O conjunto de imagens citado acima apresenta uma diversidade signi cativa de espécies orestais. Das 112 espécies, 37 são do tipo Conífera e 75 são do tipo Folhosa. Estas 112 espécies também podem ser divididas em 85 gêneros e 30 famílias. Deste modo, o uso desta base de imagens nos permite desenvolver um sistema de reconhecimento sem preocupar-se, neste momento, com a captura de imagens. Todavia, planeja-se contribuir para o enriquecimento desta base por meio da inclusão de novas espécies futuramente, trabalhando em conjunto com o departamento de Engenharia Ambiental da UFPR. Na Figura 3 apresentamos alguns exemplos de imagens extraídas da base de Martins et al [2012].

Nestes exemplos é possível notar que existe uma certa semelhança entre os padrões de textura de espécies pertencentes ao mesmo tipo (Conífera ou Folhosa). Mas é possível notar também uma certa distinção entre os padrões de tipos diferentes

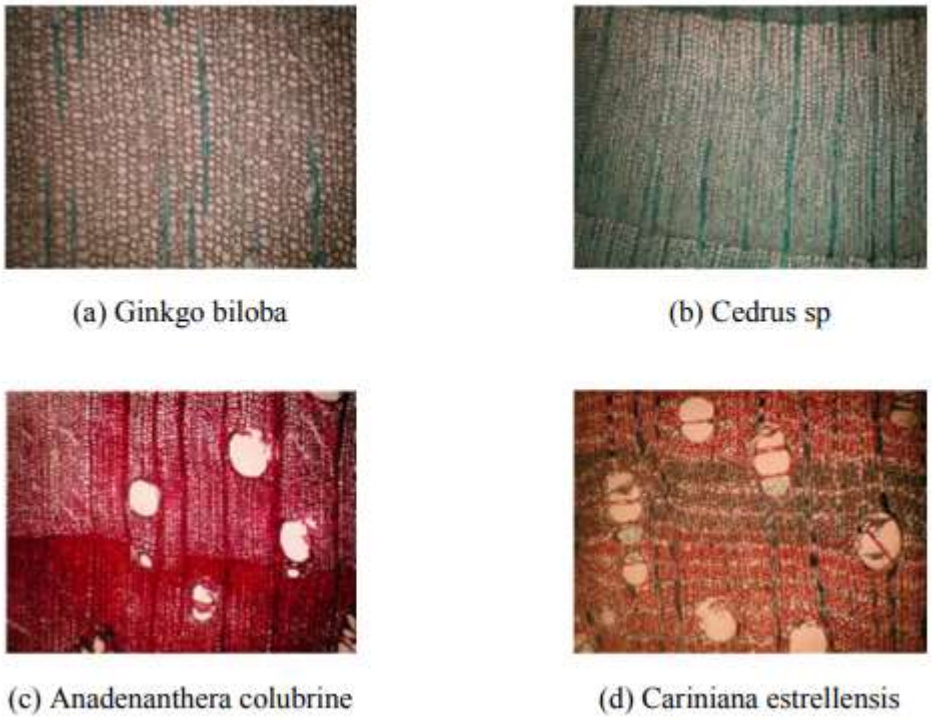



Figura 3: Amostras de espécies florestais que constituem a base de imagens.

## RESULTADOS

Na Tabela 1 tem-se uma lista de resultados envolvendo métodos de extração e com diferentes técnicas de conjunto de classificadores. Primeiramente, considerou-se as técnicas de extração GLCM, LPQ, LBP utilizando um sistema de divisão da imagens em quadrantes e combinando seus resultados para a decisão.

Logo após, utilizou-se técnicas de geração de conjuntos de classificadores (Bagging e Random Subspaces) com cada uma das técnicas de extração de características mencionadas. Finalmente, aplicou-se a técnicas de aprendizagem profunda considerando a metodologia apresentada na Seção 2. Foram testadas diversas configurações, entretanto experimentos com os parâmetros para a arquitetura da rede neural convolutiva seguir produziram os melhores resultados: 1) Camada de entrada: Entradas de 640x640 pixels e 1 canal (nível de cinza); 2) Duas combinações de camadas convolucionais e agrupadas: cada camada convolucional tem 32 filtros com tamanho ×5 - e stride definido como 1, e as camadas de pool consistem de janelas com tamanho 3 × 3 e stride 1; 3) Camada conectada localmente: - filtros de tamanho 128; 4) Camada de saída totalmente conectada (112). Com base nos resultados, observa-se que as técnicas de Random Subspaces e Deep Learning

alcançaram os melhores resultados. Entretanto, além de obter a melhor performance, a técnica de Deep Learning (CNN) utiliza apenas um classificador em comparação com a utilização de múltiplos classificadores.

| **Técnica de Extração de Características** | **1-NN** |
|---|---|
| LPQ - Uniform window- (16) | 82,96 (1,26) |
| LPQ - Gaussian window - (16) | 81,92 (1,46) |
| LPQU+LPQG+LPQB | 87.05 (0.95) |
| Bagging LPQ (16 membros) | 89.90 (0.70) |
| **Random Subspaces (16 membros)** | **90.46 (0.81)** |
| **CNN** | **91.27 (0.45)** |

Tabela 1: Resultados obtidos em comparação com resultados anteriores

## CONCLUSÃO

Os resultados demonstraram que a utilização de redes neurais convolutivas (Deep learning) produziu resultados superiores para a classificação, mesmos nos casos comparados com conjunto de classificadores, demostrando que a técnica é eficiente para a extração e classificação das espécies florestais. Porém, cabe ressaltar que o tempo de treinamento e ajustes de parâmetros é lento.

## Referências

[1] Christopher M. Bishop. Pattern Recognition and Machine Learning (Information Science and Statistics). Springer-Verlag New York, Inc., Secaucus, NJ, USA, 2006.

[2] Y. Bengio, A. Courville, and P. Vincent. Representation learning: A review and new perspectives. IEEE Transactions on Pattern Analysis and Machine Intelligence, 35(8):1798–1828, 2013.

[3] Jürgen Schmidhuber. Deep learning in neural networks: An overview. Neural Networks, 61:85 – 117, 2015.

[4] Li Deng and Dong Yu. Deep learning: Methods and applications. Foundations and Trends® in Signal Processing, 7(3–4):197–387, 2014.

[5] Itamar Arel, Derek C. Rose, and Thomas P. Karnowski. Research frontier: Deep machine learning–a new frontier in artificial intelligence research. Comp. Intell. Mag., 5(4):13–18, November 2010.

[6] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. Deep learning. Book in preparation for MIT Press, 2016.

[7] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. Deep learning. Nature, 521(7553):436–444, 2015.

[8] Yoshua Bengio. Learning deep architectures for ai. Found. Trends Mach. Learn., 2(1):1–127, January 2009.

[9] Simon Haykin. Neural Networks: A Comprehensive Foundation (3rd Edition). Prentice-Hall, Inc., Upper Saddle River, NJ, USA, 2007.

[10] Kurt Hornik. Approximation capabilities of multilayer feedforward networks. Neural Networks, 4(2):251 – 257, 1991.

[11] James J. DiCarlo, Davide Zoccolan, and Nicole C. Rust. How does the brain solve visual object recognition? Neuron, 73(3):415 – 434, 2012.

[12] Hugo Larochelle, Yoshua Bengio, Jérôme Louradour, and Pascal Lamblin. Exploring strategies for training deep neural networks. J. Mach. Learn. Res., 10:1–40, June 2009.

[13] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. Nature, 323(6088):533–536, 1986.

[14] Pascal et al Vincent. Stacked denoising autoencoders: Learning useful representations in a deep network with a local denoising criterion. J. Mach. Learn. Res., 11:3371–3408, December 2010.

[15] Salah et al Rifai. Contractive Auto-Encoders: Explicit Invariance During Feature Extraction. In ICML-11, 2011.

[16] Alireza Makhzani and Brendan Frey. k-sparse autoencoders. In ICLR 2014, 2014

[17] P. Baldi and K. Hornik. Neural networks and principal component analysis: Learning from examples without local minima. Neural Netw., 2(1):53–58, January 1989.

[18] Pierre Baldi. Autoencoders, unsupervised learning, and deep architectures. In Isabelle Guyon, Gideon Dror, Vincent Lemaire, Graham W. Taylor, and Daniel L. Silver, editors, ICML Unsupervised and Transfer Learning 2011, volume 27 of JMLR Proceedings, pages 37–50. JMLR.org, 2012.

[19] Richard O. Duda, Peter E. Hart, and David G. Stork. Pattern Classification (2Nd Edition). Wiley-Interscience, 2000.

[20] G E Hinton and R R Salakhutdinov. Reducing the dimensionality of data with neural networks. Science, 313(5786):504–507, July 2006.

[21] Y. Bengio, P. Lamblin, D. Popovici, and H. Larochelle. Greedy layer-wise training of deep networks. In B. Schölkopf, J. Platt, and T. Hoffman, editors, NIPS 19, pages 153–160. MIT Press, Cambridge, MA, 2007.

[22] Dumitru Erhan and et al. Why does unsupervised pre-training help deep learning? Journal of Machine Learning Research, 11:625–660, 2010.

[23] Marc'aurelio Ranzato et al. Building high-level features using large scale unsupervised learning. In John Langford and Joelle Pineau, editors, ICML-12, pages 81–88, New York, NY, USA, 2012. ACM.

[24] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner. Gradient-based learning applied to document recognition. Proceedings of the IEEE, 86(11):2278–2324, November 1998.

[25] Matthew D. Zeiler and Rob Fergus. Visualizing and Understanding Convolutional Networks, pages 818–833. Springer International Publishing, Cham, 2014.

[26] Kevin Jarrett, Koray Kavukcuoglu, Marc'Aurelio Ranzato, and Yann LeCun. What is the best multi-stage architecture for object recognition? In ICCV 2009, pages 2146–2153. IEEE, 2009.

[27] Y. et al LeCun. Backpropagation applied to handwritten zip code recognition. Neural Comput., 1(4):541–551, December 1989.

[28] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. Imagenet classification with deep convolutional neural networks. In F. Pereira, C. J. C. Burges, L. Bottou, and K. Q. Weinberger, editors, NIPS 25, pages 1097–1105. Curran Associates, Inc., 2012.

[29] Olga Russakovsky et al. Imagenet large scale visual recognition challenge. CoRR, abs/1409.0575, 2014.

[30] P. Sermanet, K. Kavukcuoglu, S. Chintala, and Y. LeCun. Pedestrian detection with unsupervised multi-stage feature learning. In Proc. International Conference on Computer Vision and Pattern Recognition (CVPR'13). IEEE, 2013.

[31] P. Sermanet and Y. LeCun. Traffic sign recognition with multi-scale convolutional networks. In Neural Networks (IJCNN), The 2011 International Joint Conference on, pages 2809–2813, July 2011.

[32] Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg S Corrado, and Jeff Dean. Distributed representations of words and phrases and their compositionality. In C. J. C. Burges, L. Bottou, M. Welling, Z. Ghahramani, and K. Q. Weinberger, editors, NIPS 26, pages 3111–3119. Curran Associates, Inc., 2013.

[33] Tomas Mikolov, Martin Karafiát, Lukás Burget, Jan Cernocký, and Sanjeev Khudanpur. Recurrent neural network based language model. In Takao Kobayashi, Keikichi Hirose, and Satoshi Nakamura, editors, INTERSPEECH 2010, pages 1045–1048. ISCA, 2010.

[34] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. Neural Comput., 9(8):1735–1780, November 1997.

[35] Junyoung Chung, Caglar Gulcehre, Kyunghyun Cho, and Yoshua Bengio. Empirical evaluation of gated recurrent neural networks on sequence modeling. 2014.

[36] Alex Graves, Abdel-Rahman Mohamed, and Geoffrey E. Hinton. Speech recognition with deep recurrent neural networks. In IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), pages 6645–6649. IEEE, 2013.

[37] Alex Graves and Navdeep Jaitly. Towards end-to-end speech recognition with recurrent neural networks. In ICML-14, pages 1764–1772, 2014.

[38] I. Sutskever, J. Martens, and G. Hinton. Generating text with recurrent neural networks. In L. Getoor and T. Scheffer, editors, ICML-11, ICML '11, pages 1017– 1024, New York, NY, USA, June 2011. ACM.

[39] Alex Graves. Generating sequences with recurrent neural networks. CoRR, abs/1308.0850, 2013.

[40] Ilya Sutskever, Oriol Vinyals, and Quoc V. Le. Sequence to sequence learning with neural networks. In Proceedings of the 27th International Conference on Neural Information Processing Systems, NIPS'14, pages 3104–3112, Cambridge, MA, USA, 2014. MIT Press.

[41] Nitish Srivastava, Elman Mansimov, and Ruslan Salakhudinov. Unsupervised learning of video representations using lstms. In David Blei and Francis Bach, editors, ICML-15, pages 843–852. JMLR Workshop and Conference Proceedings, 2015.

[42] Karol Gregor, Ivo Danihelka, Alex Graves, Danilo Rezende, and Daan Wierstra. Draw: A recurrent neural network for image generation. In David Blei and Francis Bach, editors, ICML-15, pages 1462–1471. JMLR Workshop and Conference Proceedings, 2015.

[43] Kelvin Xu, Jimmy Ba, Ryan Kiros, Kyunghyun Cho, Aaron Courville, Ruslan Salakhudinov, Rich Zemel, and Yoshua Bengio. Show, attend and tell: Neural image caption generation with visual attention. In David Blei and Francis Bach, editors, ICML-15, pages 2048–2057. JMLR Workshop and Conference Proceedings, 2015. [44] S. Hochreiter, Y. Bengio, P. Frasconi, and J. Schmidhuber. Gradient flow in recurrent nets: the difficulty of learning long-term dependencies. In Kremer and Kolen, editors, A Field Guide to Dynamical Recurrent Neural Networks. IEEE Press, 2001.

[45] Geoffrey E. Hinton. To recognize shapes first learn to generate images. In Computational Neuroscience: Theoretical Insights into Brain Function. Elsevier, 2007.

[46] Geoffrey Hinton, Simon Osindero, and Yee-Whye Teh. A fast learning algorithm for deep belief nets. Neural Computation, 18(7):1527–1554, 2006. [47] Marc'Aurelio Ranzato, Christopher S. Poultney, Sumit Chopra, and Yann LeCun. Efficient learning of sparse representations with an energy-based model. In Bernhard Schölkopf, John C. Platt, and Thomas Hofmann, editors, NIPS 19, pages 1137– 1144. MIT Press, 2006. [48] Vinod Nair and Geoffrey E. Hinton. Rectified linear units improve restricted boltzmann machines. In

Johannes Fürnkranz and Thorsten Joachims, editors, ICML-10, pages 807–814. Omnipress, 2010.

[49] Xavier Glorot, Antoine Bordes, and Yoshua Bengio. Deep sparse rectifier neural networks. In Geoffrey J. Gordon and David B. Dunson, editors, Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics (AISTATS-11), volume 15, pages 315–323. Journal of Machine Learning Research - Workshop and Conference Proceedings, 2011.

[50] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. In 2015 IEEE International Conference on Computer Vision, ICCV 2015, Santiago, Chile, December 7-13, 2015, pages 1026–1034, 2015.

[51] Geoffrey E. Hinton, Nitish Srivastava, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Improving neural networks by preventing co-adaptation of feature detectors. CoRR, abs/1207.0580, 2012.

[52] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. J. Mach. Learn. Res., 15(1):1929–1958, January 2014.

[53] Pierre Baldi and Peter J Sadowski. Understanding dropout. In C. J. C. Burges, L. Bottou, M. Welling, Z. Ghahramani, and K. Q. Weinberger, editors, NIPS 26, pages 2814–2822. Curran Associates, Inc., 2013.

[54] Kishore Reddy Konda, Xavier Bouthillier, Roland Memisevic, and Pascal Vincent. Dropout as data augmentation. CoRR, abs/1506.08700, 2015.

[55] D. van Dyk and X. L. Meng. The art of data augmentation (with discussion). Journal of Computational and Graphical Statistics, 10:1–111, 2001.

[56] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In David Blei and Francis Bach, editors, ICML-15, pages 448–456. JMLR Workshop and Conference Proceedings, 2015.

[57] Kyoung-Su Oh and Keechul Jung. {GPU} implementation of neural networks. Pattern Recognition, 37(6):1311 – 1314, 2004.

[58] Adam Coates et al. Deep learning with cots hpc systems. In Sanjoy Dasgupta and David Mcallester, editors, ICML-13, volume 28, pages 1337–1345. JMLR Workshop and Conference Proceedings, May 2013.

[59] Fangyi Zhang, Jürgen Leitner, Michael Milford, Ben Upcroft, and Peter Corke. Towards vision-based deep reinforcement learning for robotic motion control. In Australasian Conference on Robotics and Automation 2015, Canberra, A.C.T, September 2015.

[60] David et al Silver. Mastering the game of Go with deep neural networks and tree search. Nature, 529(7587):484–489, January 2016.

[61] Volodymyr et al Mnih. Human-level control through deep reinforcement learning. Nature, 518(7540):529–533, February 2015.

[62] Jiquan Ngiam, Aditya Khosla, Mingyu Kim, Juhan Nam, Honglak Lee, and Andrew Y. Ng. Multimodal deep learning. In Lise Getoor and Tobias Scheffer, editors, ICML-11, pages 689–696. Omnipress, 2011.

[63] Andrej Karpathy and Li Fei-Fei. Deep visual-semantic alignments for generating image descriptions. In CVPR, 2014.

[64] Andrej Karpathy, Justin Johnson, and Fei-Fei Li. Visualizing and understanding recurrent networks. CoRR, abs/1506.02078, 2015.

[65] Yann N. Dauphin, Razvan Pascanu, Caglar Gulcehre, Kyunghyun Cho, Surya Ganguli, and Yoshua Bengio. Identifying and attacking the saddle point problem in high-dimensional non-convex optimization. In Z. Ghahramani, M. Welling, C. Cortes, N.d. Lawrence, and K.q. Weinberger, editors, NIPS 27, pages 2933–2941. Curran Associates, Inc., 2014.

[66] Bruno A. Olshausen and David J. Field. Sparse coding with an overcomplete basis set: A strategy employed by v1? Vision Research, 37(23):3311 – 3325, 1997.

[Ope] OpenCV. OpenCV. (consultado em 01-03-2017). url: http:// opencv.org/.